

FEKETE LYUK — BELSŐ UNIVERZUM SZIMULÁTOR

Teljes projekt dokumentáció

A Norbi-hipotézis · Fizikai terv · Szoftverarchitektúra · Tesztek · Tudományos háttér

Norbi & Claude • 2026

Tartalom

I. RÉSZ — Hipotézis és fizikai terv (1–11. fejezet)

1. A Norbi-hipotézis | 2. Fizikai alapok | 3–4. Szoftveres modulok
5. Belső univerzum részletesen | 6. UI terv | 7. Skálázható komplexitás
8–9. Validáció és hipotézis tesztelés | 10. Jövőbeli tervek | 11. Bevy 3D UI

II. RÉSZ — Szoftverarchitektúra (12–28. fejezet)

12–13. Stack és mappaszerkezet | 14. Rust OOP | 15. Unit tesztek (12 tesztfájl)
16. GitHub Actions CI/CD | 17. Tauri UI | 18. Összefoglalás
19. Hibakezelés | 20. Adatséma | 21. PyO3 határfelület
22. Checkpoint | 23. Verziók | 24. Logging | 25. Benchmarkok | 26–28. Scripts és végleges fa

III. RÉSZ — Tudományos háttér (29–35. fejezet)

29. Kutatási kérdés | 30. Formális hipotézis | 31. Null hipotézis
32. Validációs stratégia | 33. Korlátok | 34. Irodalomjegyzék | 35. Összefoglalás

I. RÉSZ — Hipotézis és fizikai terv

1. A Norbi-hipotézis — eredeti megfogalmazás

Ez a projekt egy konkrét fizikai hipotézis szimulációs vizsgálatára épül. A hipotézis saját szavaiddal:

"Amikor egy fekete lyuk keletkezik, belsejében egy ősrobbanáshoz hasonló folyamat indul el — egy belső univerzum születik. Ez a belső univerzum a fekete lyukba beesett anyagból táplálkozik: minden beeső csillag, bolygó, aszteroida újabb anyagot ad neki, és a belső univerzumban ütközések, struktúrák, bolygók keletkeznek. A belső univerzum tágul — ahogy kívülről látjuk a fekete lyuk eseményhorizontjának növekedését. A tágulás szélén lévő objektumok szétszakadnak, és a szétszakadáskor apró adatcsomagok szöknek ki — ezeket külső megfigyelőként Hawking-sugárzásnak érzékelünk."

Ez az elmélet három jól elkülönülő részből áll amelyeket a szoftver egymástól függetlenül is tud vizsgálni:

1 Belső ősrobbanás

A fekete lyuk keletkezésekor a szingularitás helyén egy kvantumvisszapattanás következik be — az összeomlott anyag egy új, belső téridőben kezd el tágulni. Ez analóg a mi ősrobbanásunkkal.

2 Anyag táplálás

Minden beeső csillag, bolygó, aszteroida átlép az eseményhorizonton és a belső univerzumba kerül. Ott szétesik és részecskéket, ütközéseket, struktúrákat gerjeszt — a belső univerzumot folyamatosan gazdagítja.

3 Hawking- sugárzás mint széli szétszakadás

A belső univerzum tágul. A szélén lévő objektumok a tágulás miatt szétszakadnak — pontosan úgy ahogy a Big Rip forgatókönyvben. A szétszakadáskor energia és információ lép ki az eseményhorizonton keresztül. Ezt látjuk kívülről Hawking-sugárzásként.

2. Fizikai alapok

2.1 A külső fekete lyuk — mit tudunk biztosan

A külső fizika teljesen levezetett és validált. Ez az a rész amit a szimuláció standard módban pontosan reprodukál:

Schwarzschild-sugár: $r_s = 2GM/c^2$
→ Ez nő ahogy anyag esik be: $\Delta r_s = 2GM_{\text{beesett}}/c^2$
→ Ezt látjuk kívülről: a fekete lyuk 'nő'

Hawking-hőmérséklet: $T_H = \hbar c^3 / (8\pi G M k_B)$

- Minél nagyobb a fekete lyuk, annál hidegebb
- A Norbi-hipotézisben ez a belső széli szétszakadás hőmérséklete

Bekenstein-Hawking entrópia: $S = A / (4l_P^2) = 4\pi r_s^2 / (4l_P^2)$

- Az entrópia arányos az eseményhorizont felszínével
- Minden beeső objektum növeli az entrópiát

Elpárlási teljesítmény: $P = \hbar c^6 / (15360\pi G^2 M^2)$

- Ez a Hawking-sugárzás által elvitt energia sebessége
- A Norbi-hipotézisben: a belső szélén szétszakadó anyag energiája

2.2 A belső dinamika — a Norbi-modell

A belső fizika a hipotézis magja. Három egymást követő fázisból áll:

1. fázis: A belső ősrobbanás

A fekete lyuk szingularitásánál a sűrűség eléri a Planck-sűrűséget. A Loop Quantum Cosmology (LQC) szerint itt kvantumvisszapattanás következik be — az összeomlás megfordul és egy új téridő tágulni kezd:

LQC módosított Friedmann-egyenlet:

$$(\dot{a}/a)^2 = (8\pi G/3) \cdot \rho \cdot (1 - \rho/\rho_P)$$

ahol $\rho_P = c^5/(\hbar G^2) \approx 5.16 \times 10^{96} \text{ kg/m}^3$ (Planck-sűrűség)

Ha $\rho \rightarrow \rho_P$: a jobb oldal $\rightarrow 0$, tehát $\dot{a} \rightarrow 0$

- Az összeomlás megáll
- Majd $\dot{a}$ előjelet vált: tágulás kezdődik
- Ez a belső ősrobbanás pillanata

A belső tágulás inflációs fázisa:

$$a(t) = a_0 \cdot e^{(H_{\text{inf}} \cdot t)}$$

ahol $a_0 \approx l_P$ (Planck-hossz), H_{inf} nagy

► Ez az egyenlet adja a belső univerzum 'születésének' matematikai alapját. Az a_0 és H_{inf} értékei a visszapattanás pillanatának fizikai állapotától függnének.

2. fázis: Beeső anyag → belső struktúrák

Minden kívülről beeső objektum átlép az eseményhorizonton. A Norbi-modellben ez azt jelenti hogy az objektum belép a belső univerzumba. Az átmenet fizikája:

Beeső objektum sajátideje az eseményhorizonton:

$$d\tau = dt \cdot \sqrt{1 - r_s/r} \rightarrow 0 \text{ ahogy } r \rightarrow r_s$$

Kívülről: az objektum 'megfagy' az eseményhorizonton

Belülről (Norbi-modell): az objektum belép a belső univerzumba

- energiája: $E = mc^2 + \text{mozgási energia}$
- impulzusa: megmarad
- ez gerjeszti a belső ütközéseket és struktúrákat

A belső univerzum energiamérlegje:
$$E_{\text{belső}}(t) = E_{\text{ősrobbanás}} + \sum E_{\text{beeső}}(t_i)$$
$$= E_0 + \int_0^t \dot{M}_{\text{beesés}}(t') \cdot c^2 dt'$$

ahol $\dot{M}_{\text{beesés}}$ = az anyagbeeső ráta (kg/s)

➤ Ez az energiamérleg adja meg hogy a belső univerzum folyamatosan gazdagodik — nem zárt rendszer, hanem nyílt: a külső fekete lyuk táplálja.

3. fázis: Széli szétszakadás → Hawking-sugárzás

A belső univerzum tágul. A tágulás sebessége a szélén meghaladja a fénysebességet (ahogy a mi univerzumunkban is). A szélén lévő objektumok a tágulás miatt szétszakadnak — ez a Big Rip mechanizmus lokálisan:

A belső tágulás szélén a tágulási sebesség:
$$v_{\text{tágulás}}(r) = H_{\text{belső}} \cdot r$$

Ha $v_{\text{tágulás}} > c$: az objektum szétszakad
→ A szétszakadáshoz szükséges energia: $E_{\text{szak}} = G \cdot m^2 / r_{\text{obj}}$
→ A felszabaduló energia az eseményhorizontra 'nyomódik'
→ Ez jelenik meg kívülről Hawking-sugárzásként

A Hawking-sugárzás teljesítménye a Norbi-modellben:
$$P_{\text{Norbi}} = \eta \cdot \rho_{\text{szél}}(t) \cdot v_{\text{szak}}(t) \cdot A_{\text{horizont}}$$

ahol:
 η = hatásfok ($0 < \eta < 1$, meghatározandó)
 $\rho_{\text{szél}}$ = anyagsűrűség a belső univerzum szélén
 v_{szak} = szétszakadási sebesség
 $A_{\text{horizont}} = 4\pi \cdot r_s^2$ (az eseményhorizont felszíne)

Összehasonlítás a standard képlettel:
$$P_{\text{standard}} = \hbar c^6 / (15360\pi G^2 M^2)$$

A szimuláció feladata: megvizsgálni hogy $P_{\text{Norbi}} \approx P_{\text{standard}}$ -e és milyen η értéknél egyeznek meg.

➤ Ez az energiamérleg-probléma Norbi féle megoldása: a szétszakadáskor felszabaduló energia arányos a belső anyagsűrűséggel és a szétszakadási sebességgel. Ha η a helyes értéken van, a két formula ugyanazt adja.

3. A szoftver célja és felépítése

3.1 Mit szimulál a szoftver?

A szoftver egy interaktív, vizuális szimulátor amely a Norbi-hipotézist teszi láthatóvá. Két szinkronizált nézete van:

BAL PANEL — Külső nézet

JOB B PANEL — Belső nézet

Fekete lyuk gravitációs tere Bolygók/csillagok pályái kívülről Eseményhorizont mint fénylő kör Hawking-sugárzás kilépő pontjai Fekete lyuk méretének növekedése Interaktív: objektum lerakható	A belső ősrobbanástól indul Belső anyag és struktúrák fejlődése Belső objektumok megjelenése belülről Ütközések, gerjesztések vizuálisan A belső szélen szétszakadó anyag Kiszakadó adatcsomagok animációja
---	--

3.2 Időbeli szinkronizáció

A két panel időben szinkronban fut, de különböző időskálákon. A külső panel valós fizikai időt mutat (gyorsítva). A belső panel a belső univerzum saját idejét mutatja, ami az LQC szerint teljesen független a külső időtől:

```
Külső idő (t_külső): a fekete lyuk kora
→ 1 szimulációs lépés = Δt_külső (beállítható)

Belső idő (t_belső): a belső univerzum saját kora
→ Kezdetben: t_belső = 0 (az ősrobbanás pillanata)
→ Függ a belső Hubble-paramétertől: dt_belső ∝ 1/H_belső

Összekötő kapocs:
Amikor t_külső-nél egy objektum beesik,
t_belső időn megjelenik a belső univerzumban.
A két idő aránya: t_belső / t_külső = f(H_belső, r_s)
→ Ezt a szimulációs motor határozza meg lépésenként
```

4. Szoftveres terv — modulok

4.1 A modulok és felelősségeik

Modul	Felelősség
ExternalUniverse	A külső fekete lyuk fizikája: gravitációs tér, pályaszámítás, eseményhorizont, Hawking-sugárzás
ObjectSpawner	Objektumok létrehozása (felhasználó által): tömeg, méret, anyagösszetétel, kezdeti pozíció és sebesség
InfallEngine	A beesési folyamat szimulálása: geodéziaegyenlet, idődilatáció, horizonton való átlépés
InternalUniverse	A belső univerzum dinamikája: LQC tágulás, belső objektumok, ütközések, struktúrák
EdgeBreakup	A belső széli szétszakadás: mikor szakad szét egy objektum, mekkora energiát szabadít fel
HawkingBridge	Az összekötő modul: a szétszakadásból számítja a kijövő Hawking-sugárzást

Renderer	A vizuális megjelenítés: külső panel (2D gravitációs tér), belső panel (belső univerzum animáció)
PhysicsValidator	A 7 validációs teszt futtatása minden szimulációs lépésnél — ha valami megsérti a fizikát, jelez

4.2 Az adatfolyam

```

KÜLSŐ ESEMÉNY (felhasználó lerak egy bolygót)
↓
ObjectSpawner.create(mass, radius, position, velocity)
↓
ExternalUniverse.add_object(obj)
→ gravitációs pálya számítása (Schwarzschild-geodézia)
→ pálya megjelenítése a bal panelen
↓
InfallEngine.check_horizon_crossing(obj)
→ Ha obj.radius <= r_s: HORIZONTON ÁTLÉPÉS
↓
InternalUniverse.absorb(obj)
→ Az objektum megjelenik a jobb panelen
→ Ütközések és gerjesztések számítása
→ Belső energia frissítése
↓
EdgeBreakup.check_all_internal_objects()
→ Ha  $H_{\text{belső}} \cdot r_{\text{obj}} > c$ : SZÉTSZAKADÁS
→ Energia felszabadul
↓
HawkingBridge.emit(energy, position_on_horizon)
→ Hawking-sugárzás megjelenik a bal panelen
→ Fekete lyuk tömege csökken:  $\Delta M = -E/c^2$ 
↓
ExternalUniverse.update_black_hole()
→  $r_s$  frissítése
→ Bal panel frissítése

```

5. A belső univerzum modellje részletesen

5.1 Belső objektumok — mit látunk a jobb panelen

Minden kívülről beeső objektum a belső univerzumban jelenik meg. A megjelenítés fizikailag motivált: az objektum tömege, mérete és sebessége megmarad, de a belső koordináta-rendszerbe transzformálva.

```

class InternalObject:
    mass:          f64      # kg — megmarad
    radius:         f64      # m — megmarad
    position:       Vec3     # belső koordináták
    velocity:       Vec3     # belső sebesség

```

```

material:      MaterialType # kő, gáz, fém, jég...
entry_time:    f64          # mikor lépett be
fragmented:    bool         # szétszakadt-e már

# Az objektum belső pozíciója az átlépés után:
# A belső univerzum a visszapattanás pillanatától tágul
# Az átlépéskor az objektum a horizonton 'belül' jelenik meg
# Pozíciója arányos az átlépési irányával és a belső tágulással

fn transform_to_internal(external_obj: &Object,
                        crossing_time: f64,
                        internal_universe: &InternalUniverse) ->
InternalObject {
    let scale = internal_universe.scale_factor(crossing_time);
    let pos    = external_obj.direction_to_horizon * scale * 0.1;
    InternalObject { mass: external_obj.mass, position: pos, ... }
}

```

5.2 Ütközések és struktúrák

A belső objektumok gravitációsan hatnak egymásra. Az n-test gravitációs probléma numerikusan megoldható — ez a belső panel animációjának fizikai alapja:

```

# N-test gravitáció a belső univerzumban
# Minden objektumpárra:  $F = G \cdot m_1 \cdot m_2 / r^2$ 

fn compute_internal_gravity(objects: &[InternalObject]) -> Vec<Vec3> {
    let mut forces = vec![Vec3::ZERO; objects.len()];
    for i in 0..objects.len() {
        for j in (i+1)..objects.len() {
            let r    = objects[j].position - objects[i].position;
            let r_mag = r.length();
            if r_mag < SOFTENING { continue; } // ütközés elkerülés
            let f_mag = G * objects[i].mass * objects[j].mass /
r_mag.powi(2);
            let f_dir = r.normalize();
            forces[i] += f_mag * f_dir;
            forces[j] -= f_mag * f_dir;
        }
    }
    forces
}

# Ha két objektum közelebb kerül mint r_merge:
# → összeolvadnak (gravitációs összeomlás)
# → az egyesített tömeg: m_new = m1 + m2
# → ez struktúraképződést szimulál

```

5.3 A széli szétszakadás feltétele

Ez a Norbi-hipotézis kulcsmozzanata. Egy belső objektum akkor szakad szét, ha a belső tágulás okozta húzóerő meghaladja az objektum saját gravitációs kötési energiáját:

```

# Szétszakadási feltétel:
#  $H_{\text{belső}} \cdot r_{\text{obj}} > c \rightarrow$  tágulási sebesség a szélén  $> c$ 
# ahol  $r_{\text{obj}}$  = az objektum távolsága a belső univerzum középpontjától

# Pontosabban: a tidal force (árapály erő) meghaladja
# az öngravitáció kötési energiáját

# Kötési energia:  $E_{\text{bind}} = 3GM^2/(5R)$ 
# Tidal erő energiája:  $E_{\text{tidal}} = m \cdot H^2 \cdot r^2 / 2$ 

fn check_breakup(obj: &InternalObject,
                 universe: &InternalUniverse) -> Option<BreakupEvent> {
    let H = universe.hubble_parameter();
    let r = obj.position.length();
    let v_expansion = H * r;

    // Tidal energia vs kötési energia
    let e_tidal = 0.5 * obj.mass * H.powi(2) * r.powi(2);
    let e_bind = 3.0 * G * obj.mass.powi(2) / (5.0 * obj.radius);

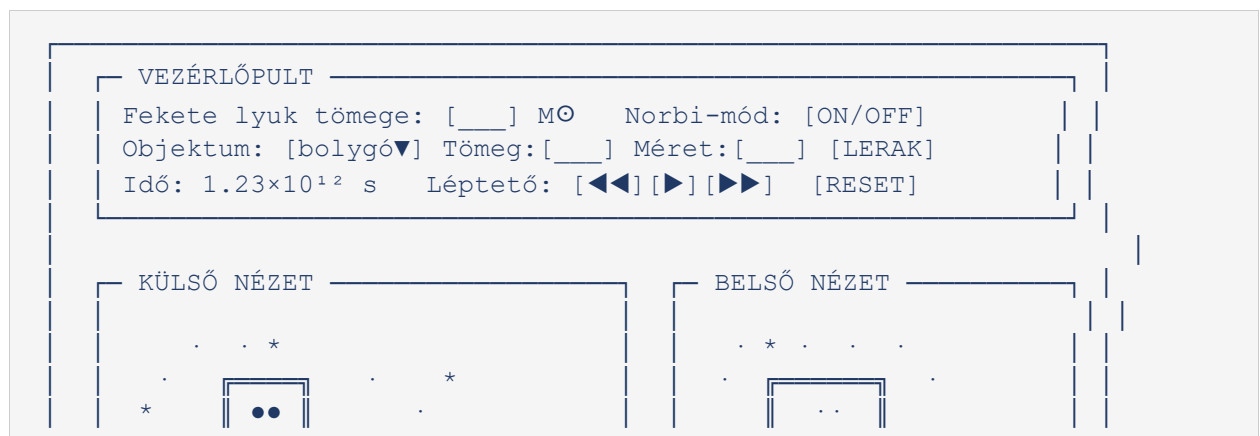
    if e_tidal > e_bind {
        Some(BreakupEvent {
            object: obj.clone(),
            released_energy: e_tidal - e_bind,
            position_on_horizon: project_to_horizon(obj.position),
        })
    } else {
        None
    }
}

```

► A *released_energy* az a energia ami az eseményhorizonton kilép és Hawking-sugárzásként jelenik meg kívülről. A *project_to_horizon* függvény leképezi a belső pozíciót az eseményhorizont megfelelő pontjára.

6. UI terv — osztott képernyő

6.1 A képernyő elrendezése





6.2 A külső panel fizikája

A bal panelen egy 2D gravitációs tér látható. A fekete lyuk középen van, körülötte a térgörbületet gravitációs mező vonalak jelzik. Az objektumok valós Schwarzschild-geodézia alapján mozognak:

```
// ExternalRenderer – bal panel

// Gravitációs mező vonalak:
// Minden vonalat a potenciál gradiens mentén húzunk
//  $\Phi(r) = -GM/r \rightarrow \nabla\Phi = GM/r^2$  (sugárirányban befelé)

// Objektum pálya (Schwarzschild-geodézia):
//  $d^2r/d\tau^2 = -GM/r^2 \cdot (1 - r_s/r) + L^2/r^3 \cdot (1 - r_s/r)$ 
// ahol L = szögmozgás-impulzus (mozgásállandó)
//  $\tau$  = sajátidő

// Az eseményhorizont: fénylő narancssárga kör,  $r = r_s$ 
// Hawking-sugárzás: apró fénylő pontok az eseményhorizonton
//  $\rightarrow$  ott jelennek meg ahol a belső szétszakadás történt

// Idődilatáció vizualizáció:
// Ahogy egy objektum közeledik  $r_s$ -hez,
// a mozgása lassul a képernyőn (idődilatáció hatása)
// de a belső panelen normál sebességgel mozog tovább
```

6.3 A belső panel fizikája

A jobb panelen a belső univerzum látható. A nézet a belső ősrobbanástól indul és az idő előrehaladtával tágul. Az objektumok a belső gravitáció szerint mozognak:

```
// InternalRenderer – jobb panel

// A belső panel 'kamera' mindig a belső univerzum közepén van
// A belső tágulást a háttér skálázódásával jelezzük
// (a háttér csillagok egyre távolabb kerülnek)

// Belső objektum megjelenítése:
// 1. Horizonton átlép  $t_{\text{külső}}$  időpontban
```

```
// 2. t_belső(t_külső) időn megjelenik a jobb panelen
// 3. Mozgása: belső N-test gravitáció szerint

// Szétszakadás animáció:
// Ha e_tidal > e_bind:
//   → az objektum fragmentekre bomlik (részecske effekt)
//   → a fragmentek kirepülnek a szél felé
//   → a szélén fénylő flash jelzi a Hawking-sugárzást
//   → EGYSZERRE a bal panelen is megjelenik a Hawking-pont

// Skálázás a számítási kapacitáshoz:
// Maximális belső objektumszám: config.max_internal_objects
// Ha megtelik: a legkisebb energiájú objektumok összevonódnak
```

7. Skálázható komplexitás

7.1 Három futási mód

Mód	Max objektumok
Lite	10 külső, 20 belső
Standard	100 külső, 200 belső
Research	1000 külső, 5000 belső

7.2 Számítási kapacitás kezelése

```
// config.rs
pub struct SimulationConfig {
    pub mode: SimMode, // Lite / Standard /
    Research
    pub max_external_objects: usize, // külső max
    pub max_internal_objects: usize, // belső max
    pub internal_dt: f64, // belső időlépés
    pub external_dt: f64, // külső időlépés
    pub norbi_mode: bool, // Norbi-hipotézis BE/KI
    pub gravitational_softening: f64, // ütközés elkerülés
}

impl SimulationConfig {
    pub fn lite() -> Self {
        Self { mode: SimMode::Lite,
            max_external_objects: 10,
            max_internal_objects: 20,
            internal_dt: 1e-10, external_dt: 1e6,
            norbi_mode: true, gravitational_softening: 0.01 }
    }
    pub fn standard() -> Self {
```

```
Self { max_external_objects: 100,
      max_internal_objects: 200, ... }
}
```

8. Validációs tesztek — mi alapján tudjuk hogy helyes?

8.1 Külső fizika validációja (7 ismert eredmény)

Teszt	Elvárt érték
$r_s(M_{\text{Nap}})$	$\approx 2954 \text{ m}$
$T_H(M_{\text{Nap}})$	$\approx 6.17 \times 10^{-8} \text{ K}$
$t_{\text{evap}} \propto M^3$	$2\times \text{ tömeg} \rightarrow 8\times \text{ idő}$
Spektrum csúcs	$v_{\text{max}} = 5.879 \times 10^{10} \cdot T_H$
$S \propto M^2$	$2\times \text{ tömeg} \rightarrow 4\times \text{ entrópia}$
Page-görbe	Csúcs $t_{\text{Page}} \approx t_{\text{evap}}/2$ -nél
LQC visszapattanás	$H^2 = 0$ ha $\rho = \rho_P$

8.2 Belső fizika validációja (Norbi-specifikus tesztek)

Teszt	Mit ellenőriz?
Energiamegmaradás	$E_{\text{belső}}(t) = E_0 + \Sigma E_{\text{beeső}}$ — a belső energia soha nem csökkent spontán
Pozitív táglulás	$H_{\text{belső}} > 0$ a visszapattanás után mindig
Szétszakadás küszöb	Ha $e_{\text{tidal}} < e_{\text{bind}}$: nincs szétszakadás. Ha $e_{\text{tidal}} > e_{\text{bind}}$: van.
Hawking konzisztencia	$P_{\text{Norbi}} \approx P_{\text{standard}}$ megfelelő η értéknél
Objektum megmaradás	Beeső objektum tömege megjelenik belül — semmi sem vész el
Időszinkron	$t_{\text{belső}} / t_{\text{külső}}$ arány stabilan tartja magát

9. A hipotézis tesztelése — mit mondanak az eredmények?

9.1 Az η hatások meghatározása

A szimulációs eredmény legfontosabb száma az η hatásfok — az az arány amely megmondja mekkora részét látjuk kívülről a belső szétszakadás energiájának:

H_1 : $\eta > 0 \rightarrow$ A Norbi-modell konzisztens:
Van olyan η érték amelynél $P_{\text{Norbi}} = P_{\text{standard}}$
Ez azt jelenti: a belső szétszakadás elvben magyarázhatja
a Hawking-sugárzást

H_0 : Nincs olyan $\eta > 0$ amely konzisztenssé teszi a modellt
 $\rightarrow$ A két teljesítmény soha nem egyezik meg
 $\rightarrow$ A Norbi-modell az adott keretrendszerben nem tartható

Szenzitivitás-analízis:
 η vizsgálata a következő paraméterek függvényében:
- Fekete lyuk tömege (M)
- Belső anyag sűrűsége
- Belső Hubble-paraméter ($H_{\text{belső}}$)
- Objektumok száma a szélén

9.2 Amit a szimuláció megmutat — és amit nem

Megmutat	Nem mutat meg
Az η konzisztens-e a standard Hawking-képlettel	Hogy a hipotézis fizikailag igaz-e
A belső dinamika stabil-e (nem omlik össze)	Hogy a valódi fekete lyukak így működnek-e
Az energiamérleg zárt-e	Hogy az AdS/CFT eredmények de Sitter térben is érvényesek
A szétszakadás szimulálható-e a rendelkezésre álló kapacitással	Hogy Kerr-fekete lyukakra is igaz-e (v3)

10. Jövőbeli tervek — v3.0

Fejlesztés	Mit ad hozzá?
Kerr-metrika (forgó fekete lyuk)	A valóságban minden fekete lyuk forog — a belső dinamika eltér
Island formula integrálása	Pontosabb Page-görbe és entrópia számítás
Soft Hair modell	Az horizonton tárolt információ explicit nyomon követése
Holografikus komplexitás	A belső 'bonyolódás' mérése az idő múlásával
Többtest szimulációk	Több fekete lyuk egyidejű szimulációja
Valós csillagászati adatok	Ismert fekete lyukak (Sgr A*, M87) paramétereivel futtatás

Ez a szoftver a Norbi-hipotézis első numerikus vizsgálata. Nem bizonyít, nem cáfol — de láthatóvá teszi. Ha a belső és külső fizika konzisztens marad, az önmagában eredmény.

11. Bevy 3D UI — részletes terv

11.1 Miért Bevy?

A Bevy egy Rust-natív, data-driven játékmotor. Mivel a fizikai mag (core/) szintén Rustban van, a Bevy közvetlen memóriáhozáféréssel tudja olvasni a szimulációs adatokat — nincs szerializálás, nincs JSON, nincs hálózati overhead. Ez a lehető leggyorsabb megoldás.

Szempont	Bevy előnye
Nyelv	Rust — ugyanaz mint a fizikai mag. Közvetlen adathozzáférés.
Teljesítmény	Natív bináris, nincs JavaScript VM. 10-100× gyorsabb sok részecskénél.
ECS	Entity Component System — a fizikai objektumok közvetlenül Bevy Entity-k.
3D kamera	Beépített OrbitCamera + FlyCamera — azonnal használható.
Részecske effekt	bevy_hanabi könyvtár — GPU alapú részecske rendszer szétszakadáshoz.
Ablak	Bevy saját natív ablakot nyit — Tauri a vezérlőpultot kezeli külön.

11.2 Két ablak architektúra

BEVY ABLAK (natív 3D megjelenítő):

BAL FELE

- Külső 3D nézet
- Forgatható orbit kamera
- Fekete lyuk gömb
- Objektumok pályái
- Hawking-sugárzás pontok

JOBB FELE

- Belső 3D nézet
- Szabad repülős kamera
- WASD + egér mozgás
- Belső struktúrák, ütközések
- Szétszakadás animáció

TAURI ABLAK (vezérlőpult — külön ablak):

Objektum lerakás | Paraméterek | Norbi-mód | Fizikai adatok |
Grafikonok

KOMMUNIKÁCIÓ: Rust channel (mpsc) — közvetlen memória, nulla overhead

11.3 Bevy ECS komponensek

```
// bevy-app/src/components.rs
#[derive(Component)]
pub struct ExternalObject { pub mass: f64, pub radius: f64, pub obj_type:
ObjectType }
```

```

#[derive(Component)]
pub struct BlackHoleMarker { pub schwarzschild_radius: f64, pub mass: f64
}

#[derive(Component)]
pub struct InternalObject { pub mass: f64, pub fragmented: bool, pub
entry_time: f64 }

#[derive(Component)]
pub struct HawkingEmissionPoint { pub energy: f64, pub lifetime: f32 }

#[derive(Component)]
pub struct TrailRenderer { pub points: Vec<Vec3>, pub max_points: usize }

```

11.4 Kamerák

```

// bevy-app/src/cameras.rs

// BAL PANEL – Orbit kamera a fekete lyuk körül
// Egér drag: forgatás | Scroll: zoom | Jobb klikk: pásztázás
pub fn setup_external_camera(mut commands: Commands) {
    commands.spawn((
        Camera3dBundle {
            transform: Transform::from_xyz(0.0, 5.0, 10.0)
                .looking_at(Vec3::ZERO, Vec3::Y),
            camera: Camera { order: 0, ..default() },
            ..default()
        },
        OrbitCamera { target: Vec3::ZERO, radius: 10.0, sensitivity: 0.5
    },
        ExternalCameraMarker,
    ));
}

// JOBB PANEL – Szabad repülő kamera (mint egy space játékban)
// WASD: mozgás | QE: fel-le | Egér: nézési irány
pub fn setup_internal_camera(mut commands: Commands) {
    commands.spawn((
        Camera3dBundle {
            camera: Camera {
                order: 1,
                viewport: Some(Viewport {
                    physical_position: UVec2::new(SCREEN_W/2, 0),
                    physical_size: UVec2::new(SCREEN_W/2, SCREEN_H),
                    ..default()
                }),
            },
            ..default()
        },
        FlyCamera { speed: 5.0, sensitivity: 0.3 },
        InternalCameraMarker,
    ));
}

```

```

    ));
}

```

11.5 Objektum lerakás — felhasználói interakció

```

// bevy-app/src/systems/input.rs
pub fn handle_object_placement(
    mouse: Res<ButtonInput<MouseButton>>,
    windows: Query<&Window>,
    camera_q: Query<(&Camera, &GlobalTransform)>,
    With<ExternalCameraMarker>>,
    mut sim: ResMut<SimulationState>,
    spawn_config: Res<SpawnConfig>,
) {
    if !mouse.just_pressed(MouseButton::Left) { return; }
    if !spawn_config.ready_to_place { return; }
    let window = windows.single();
    let (camera, cam_transform) = camera_q.single();
    if let Some(cursor) = window.cursor_position() {
        if let Some(ray) = camera.viewport_to_world(cam_transform,
            cursor) {
            let t = -ray.origin.y / ray.direction.y;
            let world_pos = ray.origin + ray.direction * t;
            // Automatikus keringési sebesség: v = sqrt(GM/r)
            let r = world_pos.length() as f64;
            let v = (G * sim.black_hole_mass() / r).sqrt();
            let vel = DVec3::new(-world_pos.z as f64, 0.0, world_pos.x as
f64)
                .normalize() * v;
            sim.add_external_object(ExternalObject {
                mass: spawn_config.mass, radius: spawn_config.radius,
                position: world_pos.as_dvec3(), velocity: vel,
                obj_type: spawn_config.obj_type,
            });
        }
    }
}

```

11.6 Vizuális elemek

Elem	Technológia
Fekete lyuk gömb	Bevy PBR + shader
Eseményhorizont	Emissive ring
Pályák	Bevy Gizmos
Beeső objektum	Bevy PBR gömb
Hawking pont	Emissive sphere
Belső ősrobbanás	bevy_hanabi

Szétszakadás	bevy_hanabi
Gravitációs tér	Gizmos grid

11.7 Bevy Cargo.toml

```
# bevy-app/Cargo.toml
[dependencies]
black-hole-core      = { path = "../core" } # saját fizikai mag
bevy                 = { version = "0.13", features = ["dynamic_linking"] }
bevy_hanabi          = "0.11"             # GPU részecske effekt
smooth-bevy-cameras = "0.11"             # OrbitCamera + FlyCamera
bevy_egui            = "0.27"             # Debug panel
```

11.8 Frissített mappaszerkezet

```
black-hole-simulator/
├── core/                  # Rust fizikai mag
├── python/               # Python elemzés
├── tauri-app/            # Vezérlőpult ablak (React + Tauri)
│   └── src/components/
│       ├── ControlPanel.tsx # Objektum lerakás, paraméterek
│       ├── DataPanel.tsx   # Fizikai adatok, grafikonok
│       └── NorbiToggle.tsx  # Norbi-mód kapcsoló
├── bevy-app/            # 3D vizualizáció (Bevy + Rust) [ÚJ]
│   ├── Cargo.toml
│   └── src/
│       ├── main.rs
│       ├── components.rs
│       ├── cameras.rs
│       ├── materials.rs
│       ├── bridge.rs       # Kommunikáció a core/-ral
│       └── systems/
│           ├── external.rs  # Külső nézet
│           ├── internal.rs  # Belső nézet
│           ├── hawking.rs   # Hawking pontok
│           ├── breakup.rs   # Szétszakadás animáció
│           └── input.rs     # Felhasználói input
├── scripts/
└── .github/workflows/
```

A Bevy 3D szimulátor teszi a Norbi-hipotézist vizuálisan megragadhatóvá: látod kívülről ahogy a bolygó bezuhan, és belülről ahogy megjelenik, mozog, szétszakad — és abban a pillanatban a külső nézeten megjelenik egy Hawking-sugárzás pont. Ez az összekötő pillanat.

II. RÉSZ — Szoftverarchitektúra

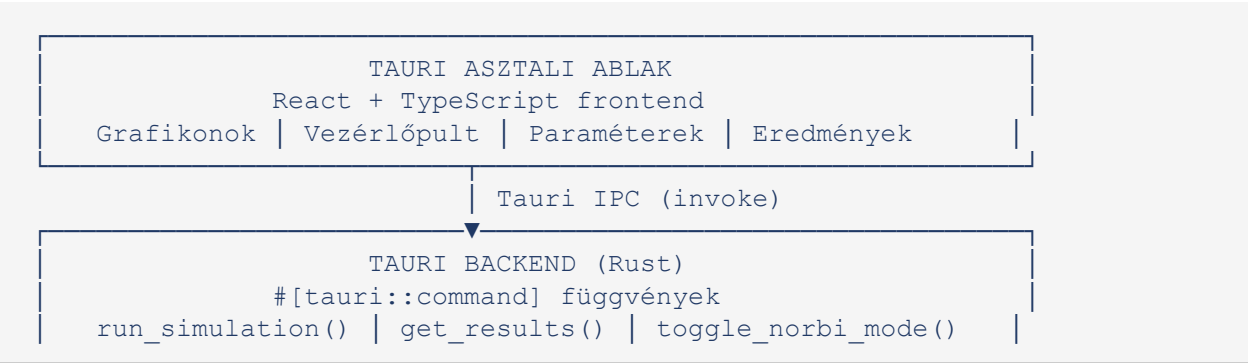
12. Technológiai stack — Rust + Python + Tauri

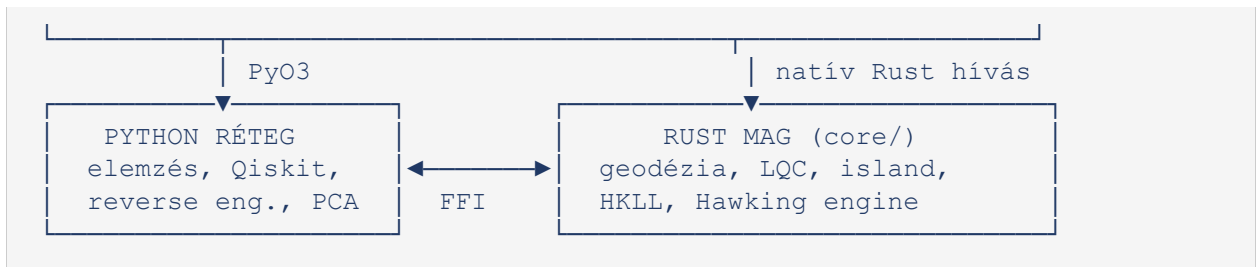
12.1 Miért ez a három?

A projekt három jól elkülönülő réteget igényel — és mindháromhoz más technológia a legalkalmasabb:

RUST mag	Szerepe: a számításigényes fizikai számítások — geodéziaintegráció, island extremalizálás, HKLL kernel, LQC egyenlet. Miért Rust: 10-100x gyorsabb Pythonnál tight loop-okban. Memóriabiztonság fordítási időben. A Tauri asztali keretrendszer szintén Rustban van — közös nyelv. Python kapcsolat: PyO3 könyvtáron keresztül. A Python úgy hívja a Rust kódot mint egy normál Python modult.
PYTHON elemzés	Szerepe: modulok összefogása, konfigurációkezelés, adatelemzés, reverse engineering, Qiskit kvantum szimuláció. Miért Python: a tudományos könyvtárak (NumPy, SciPy, Qiskit) itt a legjobbak. Te már ismered. Gyors prototipizálás. Rust kapcsolat: a nehéz számításokat Rust végzi, Python csak koordinál és elemez.
TAURI asztali UI	Szerepe: natív asztali ablak, interaktív grafikonok, szimuláció vezérlőpult, valós idejű megjelenítés. Miért Tauri: Rustban van írva (mint a magunk), a UI React + TypeScript. Sokkal kisebb és gyorsabb mint Electron. Natív ablak minden OS-en. Backend: a Tauri parancsok (commands) közvetlenül hívják a Rust magot — nincs köztes réteg.

12.2 A három réteg kommunikációja





13. Mappaszerkezet

13.1 A teljes projekt fa

```
black-hole-simulator/
├── Cargo.toml                # Rust workspace konfiguráció
├── pyproject.toml            # Python projekt konfiguráció (uv/pip)
├── package.json              # Node/Tauri frontend konfiguráció
├── .github/
│   └── workflows/
│       ├── rust-tests.yml    # Rust unit tesztek CI
│       ├── python-tests.yml  # Python unit tesztek CI
│       └── integration.yml    # Teljes integrációs teszt CI
├── core/                     # RUST MAG
│   ├── Cargo.toml
│   └── src/
│       ├── lib.rs            # Publikus API – Python és Tauri felé
│       └── black_hole/
│           ├── mod.rs        # BlackHole struct és trait-ek
│           ├── schwarzschild.rs # Schwarzschild metrika és sugárzás
│           ├── kerr.rs        # Kerr metrika (v3)
│           └── thermodynamics.rs # Hawking-hőmérséklet, entrópia
│       ├── interior/
│           ├── mod.rs
│           ├── standard.rs    # StandardInterior – geodézia
│           ├── norbi.rs       # NorbiInterior – LQC + bébiuniverzum
│           ├── baby_universe.rs # BabyUniverse struct és dinamika
│           └── cauchy.rs      # Cauchy-horizont (v3)
│       ├── radiation/
│           ├── mod.rs
│           ├── hawking_engine.rs # Hawking-sugárzás motor
│           ├── spectrum.rs     # Planck-spektrum, greybody faktor
│           └── soft_hair.rs    # Soft Hair modul (v3)
│       ├── quantum/
│           ├── mod.rs
│           ├── lqc.rs         # Loop Quantum Cosmology egyenletek
│           ├── island.rs      # Island formula (v3)
│           └── complexity.rs  # Holografikus komplexitás (v3)
│       └── time_evolution/
│           ├── mod.rs
│           └── integrator.rs  # RK45 adaptív integráló
```

```

└─ tests/                                # Rust unit tesztek
    └─ test_schwarzschild.rs
    └─ test_hawking.rs
    └─ test_lqc.rs
    └─ test_interior.rs
    └─ test_spectrum.rs

python/                                  # PYTHON ELEMZŐ RÉTEG
└─ __init__.py
└─ config.py                            # SimulationConfig osztály
└─ information_packet.py                # InformationPacket, qubit kódolás
└─ reverse_engineer.py                 # ReverseEngineer, PCA, FFT
└─ comparator.py                       # Comparator – két mód összehasonlítása
└─ qec_model.py                        # HKLL rekonstrukció (v3)
└─ quantum_sim.py                      # Qiskit integráció
└─ tests/                              # Python unit tesztek
    └─ test_information.py
    └─ test_reverse_eng.py
    └─ test_comparator.py

tauri-app/                              # TAURI ASZTALI UI
└─ src-tauri/
    └─ Cargo.toml                      # Tauri Rust konfiguráció
    └─ tauri.conf.json                 # Ablak, jogosultságok beállítása
    └─ src/
        └─ main.rs                     # Tauri belépési pont
        └─ commands.rs                 # #[tauri::command] hid a core felé
└─ src/                                # React frontend
    └─ App.tsx
    └─ components/
        └─ Dashboard.tsx               # Főképernyő
        └─ ConfigPanel.tsx             # Szimuláció paraméterek
        └─ InteriorView.tsx            # Belső állapot vizualizáció
        └─ SpectrumChart.tsx           # Sugárzási spektrum grafikon
        └─ EntropyPlot.tsx             # Entrópia és komplexitás görbe
        └─ KruskalDiagram.tsx          # Kruskal-diagram
        └─ ResultsPanel.tsx            # Reverse engineering eredmények
    └─ hooks/
        └─ useSimulation.ts            # Szimuláció állapotkezelés

docs/                                   # Dokumentáció
└─ physics/                            # Fizikai levezetések
└─ api/                                # API referencia

```

14. Rust mag — OOP struktúra

14.1 A Rust és az OOP

A Rust nem klasszikus OOP nyelv — nincs öröklés. Helyette trait-eket (interfészeket) és struct-okat (adatstruktúrákat) használunk, ami valójában rugalmasabb mint a hagyományos öröklés. A projekt ezt így alkalmazza:

OOP fogalom	Rust megfelelője
Osztály	struct + impl blokk
Interfész / absztrakt osztály	trait
Öröklés	trait implementáció (composition over inheritance)
Konstruktor	new() associated function
Private mező	pub(crate) vagy privát mező
Polimorfizmus	trait object: Box<dyn BlackHoleTrait>

14.2 A fő trait-ek

```
// core/src/black_hole/mod.rs

/// Minden fekete lyuk típus ezt a trait-et implementálja
pub trait BlackHoleTrait {
    fn mass(&self) -> f64;
    fn schwarzschild_radius(&self) -> f64;
    fn hawking_temperature(&self) -> f64;
    fn bekenstein_entropy(&self) -> f64;
    fn hawking_power(&self) -> f64;
    fn evaporation_time(&self) -> f64;
    fn update_mass(&mut self, delta_m: f64);
}

/// Belső modell trait – Standard és Norbi egyaránt implementálja
pub trait InteriorModel {
    fn simulate_step(
        &mut self,
        particle: &Particle,
        black_hole: &dyn BlackHoleTrait,
        dt: f64,
    ) -> InteriorState;
    fn at_physics_boundary(&self) -> bool;
    fn radiation_spectrum(&self) -> Vec<f64>;
}

/// Sugárzási motor trait
pub trait RadiationEngine {
    fn compute_spectrum(&self, bh: &dyn BlackHoleTrait) -> Spectrum;
    fn energy_loss_rate(&self, bh: &dyn BlackHoleTrait) -> f64;
}
```

14.3 A BlackHole struct — Schwarzschild

```
// core/src/black_hole/schwarzschild.rs
use crate::constants::*; // G, c, hbar, k_B, l_P
```

```

pub struct SchwarzschildBlackHole {
    mass: f64, // kg
    initial_mass: f64, // kg – referencia
    age: f64, // s – szimulációs idő
}

impl SchwarzschildBlackHole {
    pub fn new(mass: f64) -> Self {
        Self { mass, initial_mass: mass, age: 0.0 }
    }
}

impl BlackHoleTrait for SchwarzschildBlackHole {
    fn mass(&self) -> f64 { self.mass }

    fn schwarzschild_radius(&self) -> f64 {
        //  $r_s = 2GM/c^2$ 
        2.0 * G * self.mass / (C * C)
    }

    fn hawking_temperature(&self) -> f64 {
        //  $T_H = \hbar c^3 / (8\pi G M k_B)$ 
        (HBAR * C.powi(3)) / (8.0 * PI * G * self.mass * K_B)
    }

    fn bekenstein_entropy(&self) -> f64 {
        //  $S = A / (4 l_P^2)$  ahol  $A = 4\pi r_s^2$ 
        let r_s = self.schwarzschild_radius();
        let a = 4.0 * PI * r_s.powi(2);
        a / (4.0 * L_P.powi(2))
    }

    fn hawking_power(&self) -> f64 {
        //  $P = \hbar c^6 / (15360\pi G^2 M^2)$ 
        (HBAR * C.powi(6)) / (15360.0 * PI * G.powi(2) *
self.mass.powi(2))
    }

    fn evaporation_time(&self) -> f64 {
        //  $t = 5120\pi G^2 M^3 / (\hbar c^4)$ 
        5120.0 * PI * G.powi(2) * self.initial_mass.powi(3) / (HBAR *
C.powi(4))
    }

    fn update_mass(&mut self, delta_m: f64) {
        self.mass += delta_m;
        assert!(self.mass > 0.0, "Fekete lyuk tömege nullára csökkent!");
    }
}

```

14.4 A NorbilInterior struct

```

// core/src/interior/norbi.rs

pub struct NorbiInterior {
    baby_universe: Option<BabyUniverse>,
    bounce_occurred: bool,
    edge_breakup_rate: f64,
}

impl NorbiInterior {
    pub fn new() -> Self {
        Self { baby_universe: None, bounce_occurred: false,
edge_breakup_rate: 0.0 }
    }

    fn quantum_bounce(&mut self, density: f64) -> bool {
        // LQC feltétel:  $\rho \geq \rho_{\text{Planck}} \rightarrow$  visszapattanás
        //  $(\dot{a}/a)^2 = (8\pi G/3) \cdot \rho \cdot (1 - \rho/\rho_P)$ 
        let rho_p = RHO_PLANCK;
        let h_sq = (8.0 * PI * G / 3.0) * density * (1.0 - density /
rho_p);
        // Ha  $h_{\text{sq}} \leq 0$ : a visszapattanás megtörtént
        if h_sq <= 0.0 {
            self.bounce_occurred = true;
            self.baby_universe = Some(BabyUniverse::new(density));
            return true;
        }
        false
    }
}

impl InteriorModel for NorbiInterior {
    fn simulate_step(&mut self, particle: &Particle,
        bh: &dyn BlackHoleTrait, dt: f64) -> InteriorState {
        // 1. Geodézia integrálás (azonos a standard modellel)
        let mut state = integrate_geodesic(particle, bh, dt);

        // 2. Ha Planck-sűrűség: kvantumvisszapattanás
        if state.density >= RHO_PLANCK * 0.99 {
            let bounced = self.quantum_bounce(state.density);
            if bounced {
                state.bounce_occurred = true;
                // Beeső anyag  $\rightarrow$  bébiuniverzum
                if let Some(ref mut bu) = self.baby_universe {
                    bu.absorb_energy(particle.energy);
                    self.edge_breakup_rate = bu.edge_breakup_rate();
                }
            }
        }
        state
    }

    fn radiation_spectrum(&self) -> Vec<f64> {
        // A belső tágulás szélén szétszakadó anyag spektruma
        match &self.baby_universe {
            Some(bu) => bu.edge_radiation_spectrum(),
        }
    }
}

```

```

        None      => vec![0.0; SPECTRUM_BINS],
    }
}

fn at_physics_boundary(&self) -> bool { false } // Norbi folytatja
}

```

15. Unit tesztek

15.1 Rust tesztek — `core/src/tests/`

A Rust tesztek közvetlenül a forrás mellett élnek, a `#[cfg(test)]` attribútummal jelölve. Minden fizikai képletet tesztelünk ismert értékekkel — ha a képlet helyesen van implementálva, a teszt átmegy.

`test_schwarzschild.rs` — Fizikai képletek ellenőrzése

```

// core/src/tests/test_schwarzschild.rs
#[cfg(test)]
mod tests {
    use super::*;
    use crate::black_hole::schwarzschild::SchwarzschildBlackHole;
    use crate::black_hole::BlackHoleTrait;
    use approx::assert_relative_eq; // közelítő egyenlőség lebegőpontnál

    // Napnyi tömegű fekete lyuk – ismert fizikai értékek
    const M_SUN: f64 = 1.989e30; // kg

    #[test]
    fn test_schwarzschild_radius_sun() {
        // Elvárt:  $r_s(\text{Nap}) \approx 2954 \text{ m}$ 
        let bh = SchwarzschildBlackHole::new(M_SUN);
        assert_relative_eq!(bh.schwarzschild_radius(), 2954.0, epsilon =
1.0);
    }

    #[test]
    fn test_hawking_temperature_sun() {
        // Elvárt:  $T_H(\text{Nap}) \approx 6.17 \times 10^{-8} \text{ K}$ 
        let bh = SchwarzschildBlackHole::new(M_SUN);
        assert_relative_eq!(bh.hawking_temperature(), 6.17e-8, epsilon =
1e-10);
    }

    #[test]
    fn test_bekenstein_entropy_positive() {
        let bh = SchwarzschildBlackHole::new(M_SUN);
        assert!(bh.bekenstein_entropy() > 0.0);
    }
}

```

```

#[test]
fn test_entropy_scales_with_mass_squared() {
    //  $S \propto M^2$  – ha kétszeres a tömeg, négyszeres az entrópia
    let bh1 = SchwarzschildBlackHole::new(M_SUN);
    let bh2 = SchwarzschildBlackHole::new(2.0 * M_SUN);
    let ratio = bh2.bekenstein_entropy() / bh1.bekenstein_entropy();
    assert_relative_eq!(ratio, 4.0, epsilon = 0.001);
}

#[test]
fn test_evaporation_time_cubic_mass() {
    //  $t_{\text{evap}} \propto M^3$  – kétszeres tömeg → nyolcszoros elpárlási idő
    let bh1 = SchwarzschildBlackHole::new(1e10);
    let bh2 = SchwarzschildBlackHole::new(2e10);
    let ratio = bh2.evaporation_time() / bh1.evaporation_time();
    assert_relative_eq!(ratio, 8.0, epsilon = 0.001);
}

#[test]
fn test_larger_mass_colder() {
    // Nagyobb fekete lyuk hidegebb
    let bh_small = SchwarzschildBlackHole::new(1e10);
    let bh_large = SchwarzschildBlackHole::new(1e20);
    assert!(bh_large.hawking_temperature() <
bh_small.hawking_temperature());
}
}

```

test_lqc.rs — Kvantumvisszapattanás ellenőrzése

```

// core/src/tests/test_lqc.rs
#[cfg(test)]
mod tests {
    use super::*;
    use crate::quantum::lqc::LQCEquation;
    use crate::constants::RHO_PLANCK;

    #[test]
    fn test_hubble_zero_at_planck_density() {
        //  $(\dot{a}/a)^2 = 8\pi G\rho/3 \cdot (1 - \rho/\rho_P)$ 
        // Ha  $\rho = \rho_P$ : jobb oldal = 0 →  $\dot{a} = 0$  → visszapattanás
        let lqc = LQCEquation::new();
        let h_sq = lqc.hubble_squared(RHO_PLANCK);
        assert!(h_sq.abs() < 1e-10, "Planck-sűrűségnél  $H^2$  nullának kell
lennie");
    }

    #[test]
    fn test_normal_density_recovers_friedmann() {
        // Kis sűrűségnél ( $\rho \ll \rho_P$ ) vissza kell kapni az eredeti
képletet
        let lqc = LQCEquation::new();
        let rho = 1e10; // sokkal kisebb mint  $\rho_P$ 
    }
}

```



```

        let h_sq_lqc          = lqc.hubble_squared(rho);
        let h_sq_classical = lqc.classical_hubble_squared(rho);
        // A kettőnek közel egyenlőnek kell lennie
        let rel_diff = (h_sq_lqc - h_sq_classical).abs() /
h_sq_classical;
        assert!(rel_diff < 1e-10,
            "Kis sűrűségnél LQC  $\approx$  klasszikus Friedmann");
    }

#[test]
fn test_bounce_triggers_above_planck() {
    // Ha  $\rho \geq \rho_P$ : a visszapattanás flag igazra vált
    let mut interior = NorbiInterior::new();
    let triggered = interior.quantum_bounce(RHO_PLANCK * 1.01);
    assert!(triggered, "Planck-sűrűség felett a visszapattanás kell
induljon");
}

#[test]
fn test_no_bounce_below_planck() {
    let mut interior = NorbiInterior::new();
    let triggered = interior.quantum_bounce(RHO_PLANCK * 0.5);
    assert!(!triggered, "Planck-sűrűség alatt nincs visszapattanás");
}
}

```

test_hawking.rs — Sugárzás konzisztencia

```

// core/src/tests/test_hawking.rs
#[cfg(test)]
mod tests {
    use super::*;
    use crate::radiation::hawking_engine::HawkingEngine;

    #[test]
    fn test_mass_decreases_over_time() {
        // Az elpárlás során a tömeg csökken
        let mut bh = SchwarzschildBlackHole::new(1e15);
        let engine = HawkingEngine::new();
        let m_before = bh.mass();
        engine.evolve_step(&mut bh, 1.0); // 1 másodperc
        assert!(bh.mass() < m_before, "A tömegnek csökkenie kell");
    }

    #[test]
    fn test_energy_conservation() {
        // Az elvesztett tömegenergia = kisugárzott energia
        let mut bh = SchwarzschildBlackHole::new(1e15);
        let engine = HawkingEngine::new();
        let m_before = bh.mass();
        let emitted = engine.evolve_step(&mut bh, 1.0);
        let delta_e = (m_before - bh.mass()) * C * C;
        assert_relative_eq!(emitted, delta_e, epsilon = 1e-10);
    }
}

```

```

#[test]
fn test_spectrum_peak_matches_temperature() {
    // Wien-törvény:  $\lambda_{\max} \cdot T = 2.898 \times 10^{-3} \text{ m} \cdot \text{K}$ 
    let bh = SchwarzschildBlackHole::new(1e10);
    let t_h = bh.hawking_temperature();
    let engine = HawkingEngine::new();
    let spectrum = engine.compute_spectrum(&bh);
    let peak_freq = spectrum.peak_frequency();
    // Wien törvény frekvencia formában:  $\nu_{\max} = 5.879 \times 10^{10} \cdot T$ 
    let expected_peak = 5.879e10 * t_h;
    assert_relative_eq!(peak_freq, expected_peak, epsilon = 0.05 *
expected_peak);
}

#[test]
fn test_greybody_factor_bounds() {
    // A greybody faktornak 0 és 1 között kell lennie
    let bh = SchwarzschildBlackHole::new(1e15);
    let engine = HawkingEngine::new();
    for freq in [1e10, 1e11, 1e12, 1e13_f64] {
        let gamma = engine.greybody_factor(freq, &bh);
        assert!(gamma >= 0.0 && gamma <= 1.0,
            "Greybody faktor {:.2e} Hz-en kívül [0,1]", freq);
    }
}
}

```

test_validation.rs — A 7 validációs eredmény tesztjei

Ez a tesztfájl a tudományos háttér dokumentumban leírt 7 validációs eredményt ellenőrzi. Ha bármelyik megbukik, a fizikai implementációban hiba van.

```

// core/src/tests/test_validation.rs
// A 7 kötelező validációs teszt – mind ismert analitikus eredmény
#[cfg(test)]
mod validation {
    use super::*;
    use approx::assert_relative_eq;
    const M_SUN: f64 = 1.989e30;

    // VALIDÁCIÓ 1: Schwarzschild-sugár [SCH16]
    #[test]
    fn val_01_schwarzschild_radius_sun() {
        let bh = SchwarzschildBlackHole::new(M_SUN).unwrap();
        assert_relative_eq!(bh.schwarzschild_radius(), 2954.0, epsilon =
5.0);
    }

    // VALIDÁCIÓ 2: Hawking-hőmérséklet [HAW74]
    #[test]
    fn val_02_hawking_temperature_sun() {
        let bh = SchwarzschildBlackHole::new(M_SUN).unwrap();
    }
}

```

```

        assert_relative_eq!(bh.hawking_temperature().unwrap(), 6.17e-8,
epsilon = 1e-10);
    }

    // VALIDÁCIÓ 3: Elpárlási idő köbös arány [HAW75]
    #[test]
    fn val_03_evaporation_time_cubic_in_mass() {
        let bh1 = SchwarzschildBlackHole::new(1e10).unwrap();
        let bh2 = SchwarzschildBlackHole::new(2e10).unwrap();
        let ratio = bh2.evaporation_time() / bh1.evaporation_time();
        assert_relative_eq!(ratio, 8.0, epsilon = 0.001);
    }

    // VALIDÁCIÓ 4: Wien-törvény – spektrum csúcs [PLA00]
    #[test]
    fn val_04_spectrum_peak_follows_wien_law() {
        //  $\nu_{\text{max}} = 5.879 \times 10^{10} \cdot T_H$ 
        let bh = SchwarzschildBlackHole::new(1e10).unwrap();
        let t_h = bh.hawking_temperature().unwrap();
        let spectrum =
HawkingEngine::new().compute_spectrum(&bh).unwrap();
        let peak_freq = spectrum.peak_frequency();
        let wien_peak = 5.879e10 * t_h;
        // 5% tolerancia – a diszkrét binelés okoz kis eltérést
        assert_relative_eq!(peak_freq, wien_peak, epsilon = 0.05 *
wien_peak);
    }

    // VALIDÁCIÓ 5: Entrópia négyzetes arány [BEK73]
    #[test]
    fn val_05_entropy_scales_as_mass_squared() {
        let bh1 = SchwarzschildBlackHole::new(M_SUN).unwrap();
        let bh2 = SchwarzschildBlackHole::new(2.0 * M_SUN).unwrap();
        let ratio = bh2.bekenstein_entropy() / bh1.bekenstein_entropy();
        assert_relative_eq!(ratio, 4.0, epsilon = 0.001);
    }

    // VALIDÁCIÓ 6: Page-görbe alakja [PAG93]
    #[test]
    fn val_06_page_curve_rises_then_falls() {
        // Az entrópiának először nőnie, csúcst elérnie, majd csökkenie
kell
        let bh = SchwarzschildBlackHole::new(1e15).unwrap();
        let engine = HawkingEngine::new();
        let island = IslandFormula::new();
        let mut timeline: Vec<f64> = Vec::new();
        let mut bh_sim = bh.clone();
        for _ in 0..1000 {
            let _ = engine.evolve_step(&mut bh_sim, 1e50);
            let s = island.compute_generalized_entropy(
                bh_sim.age(), &bh_sim, &RadiationState::default()
            ).unwrap();
            timeline.push(s);
        }
        // Volt csúcs és utána csökkenés?

```

```

    let peak_idx = timeline.iter().enumerate()
        .max_by(|a,b| a.1.partial_cmp(b.1).unwrap()).unwrap().0;
    assert!(peak_idx > 0 && peak_idx < timeline.len() - 1,
        "A csúcsnak a közepén kell lennie, nem a széleken");
    let after_avg = timeline[peak_idx..].iter().sum::<f64>()
        / (timeline.len() - peak_idx) as f64;
    assert!(after_avg < timeline[peak_idx],
        "A csúcs után az entrópiának csökkenie kell");
}

// VALIDÁCIÓ 7: LQC visszapattanás feltétele [ASH06]
#[test]
fn val_07_lqc_hubble_zero_at_planck_density() {
    //  $H^2 = 0$  ha  $\rho = \rho_{\text{Planck}}$ 
    let lqc = LQCEquation::new();
    let h_sq = lqc.hubble_squared(RHO_PLANCK).unwrap();
    assert!(h_sq.abs() < 1e-10,
        "Planck-sűrűségnél  $H^2 = \{h\_sq:.3e\}$ , nullának kellene");
}
}

```

test_model_comparison.rs — A két modell összehasonlítási tesztjei

Ezek a tesztek biztosítják hogy a standard és a Norbi modell pontosan ott válik el ahol kellene — a Planck-skálán — és előtte teljesen azonosan viselkednek.

```

// core/src/tests/test_model_comparison.rs
#[cfg(test)]
mod model_comparison {
    use super::*;
    use approx::assert_relative_eq;

    // ÖSSZEHASONLÍTÁS 1: Standard modell megáll Planck-határnál
    #[test]
    fn cmp_01_standard_model_stops_at_planck_scale() {
        let mut interior = StandardInterior::new();
        let particle = Particle::test_particle();
        let bh = SchwarzschildBlackHole::new(1e15).unwrap();
        let mut reached = false;
        for _ in 0..100000 {
            let state = interior.simulate_step(&particle, &bh, 1e-44);
            if state.at_planck_scale {
                assert!(interior.at_physics_boundary(),
                    "StandardInterior-nak jelezni kell a fizika
határát");
                assert!(state.physics_boundary.is_some(),
                    "PhysicsBoundary-nek beállítva kell lennie");
                reached = true;
                break;
            }
        }
        assert!(reached, "Standard modellnek Planck-határt kell
elérnie");
    }
}

```

```

// ÖSSZEHASONLÍTÁS 2: Norbi modell folytatódik Planck után
#[test]
fn cmp_02_norbi_model_continues_after_planck_scale() {
    let mut interior = NorbiInterior::new();
    let particle = Particle::test_particle();
    let bh = SchwarzschildBlackHole::new(1e15).unwrap();
    let mut bounced = false;
    for _ in 0..100000 {
        let state = interior.simulate_step(&particle, &bh, 1e-44);
        if state.bounce_occurred {
            assert!(!interior.at_physics_boundary(),
                "NorbiInterior nem áll meg a Planck-határnál");
            assert!(state.baby_universe.is_some(),
                "Visszapattanás után bébiuniverzumnak kell lennie");
            bounced = true;
            break;
        }
    }
    assert!(bounced, "Norbi modellnek visszapattanást kell
    produkálnia");
}

// ÖSSZEHASONLÍTÁS 3: Mindkét modell egyezik Planck előtt
#[test]
fn cmp_03_both_models_agree_before_planck_scale() {
    // Ez a legfontosabb teszt: csak a Planck-skálán szabad eltérni
    let mut std_int = StandardInterior::new();
    let mut norbi_int = NorbiInterior::new();
    let particle = Particle::test_particle();
    let bh = SchwarzschildBlackHole::new(1e15).unwrap();
    for step in 0..100000 {
        let std_s = std_int.simulate_step(&particle, &bh, 1e-44);
        let norbi_s = norbi_int.simulate_step(&particle, &bh, 1e-44);
        if !std_s.at_planck_scale {
            assert_relative_eq!(std_s.radius, norbi_s.radius,
                epsilon = 1e-10,
                "{}. lépésnél sugár eltér Planck előtt", step);
            assert_relative_eq!(std_s.density, norbi_s.density,
                epsilon = 1e-10,
                "{}. lépésnél sűrűség eltér Planck előtt", step);
        } else { break; }
    }
}

// ÖSSZEHASONLÍTÁS 4: Norbi sugárzás ≠ standard sugárzás
#[test]
fn cmp_04_norbi_radiation_differs_from_standard() {
    let bh = SchwarzschildBlackHole::new(1e15).unwrap();
    let spec_std =
    HawkingEngine::standard().compute_spectrum(&bh).unwrap();
    let spec_norbi =
    HawkingEngine::norbi().compute_spectrum(&bh).unwrap();
    let any_diff = spec_std.intensities.iter()
        .zip(spec_norbi.intensities.iter())

```

```

        .any(|(a, b)| (a - b).abs() > 1e-50);
        assert!(any_diff, "A két modell sugárzása azonos – Norbi nincs
implementálva");
    }

    // ÖSSZEHASONLÍTÁS 5: Bébiuniverzum energiája és tágulása pozitív
    #[test]
    fn cmp_05_baby_universe_has_positive_energy_and_expansion() {
        let mut interior = NorbiInterior::new();
        let particle = Particle { energy: 1e30,
        ..Particle::test_particle() };
        let bh = SchwarzschildBlackHole::new(1e15).unwrap();
        for _ in 0..100000 {
            let state = interior.simulate_step(&particle, &bh, 1e-44);
            if let Some(ref bu) = state.baby_universe {
                assert!(bu.total_energy > 0.0, "Bébiuniverzum
energiájának pozitívnak kell lennie");
                assert!(bu.scale_factor > 0.0, "Skálafaktornak pozitívnak
kell lennie");
                assert!(bu.expansion_rate > 0.0, "Tágulási rátának
pozitívnak kell lennie");
                break;
            }
        }
    }
}

```

15.2 Python tesztek — python/tests/

test_information.py — InformationPacket

```

# python/tests/test_information.py
import pytest
from python.information_packet import InformationPacket

class TestInformationPacket:

    def test_hash_deterministic(self):
        """Ugyanaz a payload mindig ugyanazt a hash-t adja"""
        payload = {'uzenet': 'teszt', 'szam': 42}
        p1 = InformationPacket(payload)
        p2 = InformationPacket(payload)
        assert p1.hash_sha3 == p2.hash_sha3

    def test_different_payloads_different_hash(self):
        """Különböző payloadok különböző hash-t adnak"""
        p1 = InformationPacket({'a': 1})
        p2 = InformationPacket({'a': 2})
        assert p1.hash_sha3 != p2.hash_sha3

    def test_qubit_normalization(self):
        """A qubitek normáltak:  $|\alpha|^2 + |\beta|^2 = 1$ """
        packet = InformationPacket({'teszt': True})

```

```

        for qubit in packet.qubits:
            norm = abs(qubit.alpha)**2 + abs(qubit.beta)**2
            assert abs(norm - 1.0) < 1e-10, f"Qubit normálás hibás:
{norm}"

    def test_von_neumann_entropy_positive(self):
        packet = InformationPacket({'x': list(range(100))})
        assert packet.entropy >= 0.0

    def test_entropy_zero_for_pure_state(self):
        """Tiszta állapotnál a Von Neumann entrópia nulla"""
        # Egyetlen qubit |0> állapotban: teljesen ismert → S=0
        packet = InformationPacket.__pure_state_for_test__()
        assert abs(packet.entropy) < 1e-10

```

test_comparator.py — Két mód összehasonlítása

```

# python/tests/test_comparator.py
import pytest
import numpy as np
from python.comparator import Comparator

class TestComparator:

    def test_identical_inputs_zero_difference(self):
        """Ha a két mód azonos: a spektrum különbség nulla"""
        dummy_spectrum = np.ones(1000)
        result = Comparator.compare_spectra(dummy_spectrum,
dummy_spectrum)
        assert np.allclose(result['delta_spectrum'], 0.0)
        assert result['snr'] < 1.0

    def test_detectable_difference(self):
        """Ha a különbség nagy: SNR > 3 (detektálható)"""
        s1 = np.ones(1000)
        s2 = np.ones(1000) * 2.0 # 100% különbség
        result = Comparator.compare_spectra(s1, s2)
        assert result['detectable'] == True
        assert result['snr'] > 3.0

    def test_improvement_range(self):
        """Az info visszanyerési javulás -1 és 1 között van"""
        re_std = {'similarity': 0.3}
        re_norbi = {'similarity': 0.5}
        result = Comparator.compare_information_recovery(re_std,
re_norbi)
        assert -1.0 <= result['improvement'] <= 1.0
        assert abs(result['improvement'] - 0.2) < 1e-10

    def test_interior_diverges_at_planck(self):
        """A két belső modellnek a Planck-skálán kell szétválnia"""
        result = Comparator.compare_interior_evolution(

```

```
standard_trajectory=self._make_trajectory(stops_at_planck=True),
        norbi_trajectory=self._make_trajectory(stops_at_planck=False)
    )
    assert result['agree_until_planck'] == True
```

16. GitHub Actions — CI/CD

16.1 Rust tesztek workflow

```
# .github/workflows/rust-tests.yml
name: Rust Core Tests

on:
  push:
    branches: [main, develop]
    paths: ['core/**']
  pull_request:
    branches: [main]
    paths: ['core/**']

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Rust toolchain telepítés
        uses: dtolnay/rust-toolchain@stable
        with:
          components: rustfmt, clippy

      - name: Cargo cache
        uses: Swatinem/rust-cache@v2
        with:
          workspaces: core

      - name: Formátum ellenőrzés
        run: cargo fmt --manifest-path core/Cargo.toml -- --check

      - name: Clippy (lint)
        run: cargo clippy --manifest-path core/Cargo.toml -- -D warnings

      - name: Unit tesztek futtatása
        run: cargo test --manifest-path core/Cargo.toml --verbose

      - name: Teszt lefedettség (tarpaulin)
        run: |
```



```
cargo install cargo-tarpaulin
cargo tarpaulin --manifest-path core/Cargo.toml --out Xml

- name: Lefedettség feltöltése Codecov-ra
  uses: codecov/codecov-action@v3
```

16.2 Python tesztek workflow

```
# .github/workflows/python-tests.yml
name: Python Analysis Tests

on:
  push:
    branches: [main, develop]
    paths: ['python/**']
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.11', '3.12'] # Több verzió tesztelünk

    steps:
      - uses: actions/checkout@v4

      - name: Python ${ matrix.python-version } telepítés
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }

      - name: Rust toolchain (PyO3-hoz kell)
        uses: dtolnay/rust-toolchain@stable

      - name: Függőségek telepítése
        run: |
          pip install uv
          uv pip install -e '[dev]'
          # Ez lefordítja a Rust magot is (maturin)

      - name: Ruff lint
        run: ruff check python/

      - name: Mypy típusellenőrzés
        run: mypy python/

      - name: Pytest futtatása
        run: pytest python/tests/ -v --cov=python --cov-report=xml

      - name: Codecov feltöltés
```

```
uses: codecov/codecov-action@v3
```

16.3 Integrációs teszt workflow

```
# .github/workflows/integration.yml
name: Integration Tests

on:
  push:
    branches: [main]
  schedule:
    - cron: '0 2 * * *' # Minden éjjel 2-kor automatikusan fut

jobs:
  integration:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Rust + Python telepítés
        uses: dtolnay/rust-toolchain@stable

      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Teljes projekt build
        run: |
          pip install uv maturin
          maturin develop # Rust mag fordítás + Python binding
          uv pip install -e '.[dev]'

      - name: Teljes szimuláció futtatása (Planck-tömeg)
        run: |
          python -m python.main \
            --mass 2.176e-8 \
            --norbi-mode false \
            --no-ui \
            --output ci_results_standard.json

      - name: Norbi módban is
        run: |
          python -m python.main \
            --mass 2.176e-8 \
            --norbi-mode true \
            --no-ui \
            --output ci_results_norbi.json

      - name: Eredmény validáció
        run: python scripts/validate_results.py \
          ci_results_standard.json \
```

```

        ci_results_norbi.json
# Ez ellenőrzi hogy:
# - Az entrópia nem negatív
# - A tömeg monoton csökken
# - A spektrum normált
# - A hash rekonstrukció jobb Norbi módban

- name: Eredmény archiválás
  uses: actions/upload-artifact@v4
  with:
    name: simulation-results
    path: ci_results_*.json
    retention-days: 30

```

17. Tauri asztali UI

17.1 Miért Tauri?

Szempont	Tauri vs. Electron
Méret	Tauri: ~10 MB. Electron: ~150 MB. Tauri sokkal kisebb.
Sebesség	Tauri natív webview-t használ, nem Chromium-ot. Gyorsabb indulás.
Rust integráció	A Tauri backend Rustban van — közvetlenül hívja a core/ magot.
Biztonság	Finomhangolt jogosultsági rendszer — csak amit engedélyezel, azt éri el.
Frontend	Bármilyen: React, Vue, Svelte. Mi React + TypeScript-et használunk.

17.2 A Tauri parancsok — híd a core felé

```

// tauri-app/src-tauri/src/commands.rs
use black_hole_core::*; // a core/ Rust mag importálva

/// Szimuláció indítása — React hívja, Rust futtatja
#[tauri::command]
pub async fn run_simulation(
    mass: f64,
    norbi_mode: bool,
    payload: serde_json::Value,
) -> Result<SimulationResults, String> {
    let config = SimulationConfig { mass, norbi_mode };
    let info = InformationPacket::from_json(payload)
        .map_err(|e| e.to_string())?;
    let results = Simulator::new(config).run(info)
        .map_err(|e| e.to_string())?;
    Ok(results)
}

```

```

/// Valós idejű állapot lekérése futás közben
#[tauri::command]
pub fn get_current_state() -> CurrentState {
    SIMULATION_STATE.lock().unwrap().clone()
}

/// Norbi mód kapcsoló
#[tauri::command]
pub fn toggle_norbi_mode(enabled: bool) {
    CONFIG.lock().unwrap().norbi_mode = enabled;
}

```

17.3 A React frontend főbb komponensei

```

// tauri-app/src/components/Dashboard.tsx
import { invoke } from '@tauri-apps/api/tauri'
import { useState } from 'react'
import ConfigPanel from './ConfigPanel'
import InteriorView from './InteriorView'
import SpectrumChart from './SpectrumChart'
import EntropyPlot from './EntropyPlot'
import KruskalDiagram from './KruskalDiagram'
import ResultsPanel from './ResultsPanel'

export default function Dashboard() {
  const [running, setRunning] = useState(false)
  const [results, setResults] = useState(null)
  const [norbiMode, setNorbiMode] = useState(false)

  async function startSimulation(config) {
    setRunning(true)
    // Tauri IPC hívás a Rust maghoz
    const res = await invoke('run_simulation', {
      mass: config.mass,
      norbiMode: norbiMode,
      payload: config.infoPayload,
    })
    setResults(res)
    setRunning(false)
  }

  return (
    <div className='dashboard'>
      <ConfigPanel
        norbiMode={norbiMode}
        onNorbiToggle={setNorbiMode} // ← Ez a kapcsoló
        onStart={startSimulation}
      />
      {results && <>
        <InteriorView data={results.interior} norbi={norbiMode} />
        <SpectrumChart data={results.spectrum} norbi={norbiMode} />
        <EntropyPlot data={results.entropy} />
        <KruskalDiagram data={results.kruskal} />
      </>
    </div>
  )
}

```

```
        <ResultsPanel    data={results.reverse_engineering} />
      </>
    </div>
  )
}
```

17.4 Az UI főbb képernyői

Képernyő / Panel	Mit mutat?
ConfigPanel	Tömeg csúszka, Norbi mód kapcsoló (alapból KI), információcsomag szerkesztő, Indítás gomb.
InteriorView	A fekete lyuk belseje animálva — bal: standard, jobb: Norbi. Valós időben frissül futás közben.
SpectrumChart	A Hawking-sugárzás spektruma az idő függvényében. Kettős nézet ha Norbi mód be van kapcsolva.
EntropyPlot	Bekenstein-Hawking entrópia + sugárzási entrópia + komplexitás egy grafikonon. Page-idő jelölve.
KruskalDiagram	Interaktív Kruskal-diagram — kattintható tartományok, animált részecske pálya.
ResultsPanel	Hash rekonstrukció eredménye, hasonlóság százalék, Norbi-hipotézis korreláció.

18. Teljes architektúra összefoglalás

Ez a szoftver három rétegből áll: egy Rust mag ami a fizikát számítja, egy Python elemző réteg ami az adatokat értelmezi, és egy Tauri asztali UI ami mindent megjelenít. A Norbi-hipotézis egy kapcsolóval be- és kikapcsolható — és a szoftver pontosan megmutatja mi különbözik a kettő között.

Technológia		
Réteg	Réteg	Technológia
Réteg	Technológia	Felelősség
Rust mag	Rust 1.75+, PyO3	Fizikai számítások, geodézia, LQC, Hawking-motor, spektrum
Python elemzés	Python 3.12, NumPy, SciPy, Qiskit	Reverse engineering, PCA, FFT, hash rekonstrukció, Qiskit
Tauri UI	Tauri 2.0, React 18, TypeScript	Natív asztali ablak, interaktív grafikonok, vezérlőpult
Tesztek	cargo test, pytest, GitHub Actions	Unit tesztek, integrációs tesztek, nightly CI futtatás
Build	maturin, cargo, vite	Rust → Python binding, Tauri app csomagolás

FONTOS: A szoftver komplex — de moduláris. Lehet modul-modulonként fejleszteni és tesztelni. Az ajánlott sorrend: `core/schwarzschild.rs` → `core/hawking_engine.rs` → `python/information_packet.py` → CLI futtatás → Tauri UI utoljára.

19. Hibakezelési stratégia

19.1 Numerikus határesetek — Rust szinten

Fizikai szimulációban a legtöbb hiba nem programozási hiba, hanem numerikus összeomlás. Ezeket a Rust magban kell elkapni mielőtt a Python vagy UI réteget elérik.

```
// core/src/error.rs - egységes hibatípus az egész core/-ban
#[derive(Debug, thiserror::Error)]
pub enum SimulationError {
    #[error("Fekete lyuk tömege nullára csökkent t={time:.3e}s-nél")]
    MassExhausted { time: f64 },
    #[error("Numerikus divergencia: {field} = {value:.3e} t={time:.3e}s-nél")]
    NumericalDivergence { field: String, value: f64, time: f64 },
    #[error("NaN érték keletkezett: {context}")]
    NaNDetected { context: String },
    #[error("Planck-határ elérve r={radius:.3e}m-nél")]
    PlanckBoundaryReached { radius: f64 },
    #[error("Integrálás nem konvergált {max_steps} lépés után")]
    IntegrationFailed { max_steps: usize },
    #[error("Checkpoint hiba: {0}")]
    CheckpointFailed(String),
}

// Minden számítás Result<T, SimulationError>-t ad vissza
pub fn hawking_temperature(mass: f64) -> Result<f64, SimulationError> {
    if mass <= 0.0 {
        return Err(SimulationError::MassExhausted { time: 0.0 });
    }
    let t = (HBAR * C.powi(3)) / (8.0 * PI * G * mass * K_B);
    if t.is_nan() || t.is_infinite() {
        return Err(SimulationError::NaNDetected {
            context: format!("hawking_temperature(mass={mass:.3e})"),
        });
    }
    Ok(t)
}
```

19.2 Fizikai határesetek — kötelező tesztek

```
// core/src/tests/test_edge_cases.rs
#[test]
fn test_zero_mass_returns_error() {
    assert!(SchwarzschildBlackHole::new(0.0).is_err());
}
#[test]
fn test_mass_never_negative_during_evaporation() {
    let mut bh = SchwarzschildBlackHole::new(1e8).unwrap();
    let engine = HawkingEngine::new();
    for _ in 0..10000 {
        if engine.evolve_step(&mut bh, 1.0).is_err() { break; }
        assert!(bh.mass() > 0.0);
    }
}
#[test]
fn test_nan_propagation_caught() {
    let result = hawking_temperature(f64::NAN);
    assert!(matches!(result, Err(SimulationError::NaNDetected { .. })));
}
#[test]
fn test_zero_frequency_spectrum_is_zero() {
    let bh = SchwarzschildBlackHole::new(1e15).unwrap();
    let eng = HawkingEngine::new();
    let val = eng.planck_spectrum(0.0,
bh.hawking_temperature().unwrap());
    assert_eq!(val.unwrap(), 0.0);
}
```

19.3 Pánik kezelés a PyO3 határon

```
// core/src/lib.rs – pánik sosem éri el a Python réteget
#[pyfunction]
pub fn run_simulation_py(mass: f64, norbi_mode: bool,
                        payload_json: &str) -> PyResult<String> {
    let result = std::panic::catch_unwind(|| {
        Simulator::new(SimulationConfig { mass, norbi_mode })
            .run_json(payload_json)
    });
    match result {
        Ok(Ok(json)) => Ok(json),
        Ok(Err(e))   =>
Err(PyErr::new::<pyo3::exceptions::PyRuntimeError, _>(
            format!("Szimulációs hiba: {e}"))),
        Err(_)       =>
Err(PyErr::new::<pyo3::exceptions::PyRuntimeError, _>(
            "Váratlan belső hiba (Rust panic)")),
    }
}
```

20. Teljes adatséma

```
// core/src/types.rs – összes struktúra
use serde::{Serialize, Deserialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Particle {
    pub mass: f64, pub energy: f64, pub angular_momentum: f64,
    pub initial_radius: f64, pub particle_type: ParticleType,
}
#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum ParticleType { Photon, Electron, Proton, Neutron, Custom(String)
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct InteriorState {
    pub time: f64, pub proper_time: f64, pub radius: f64,
    pub density: f64, pub ricci_scalar: f64,
    pub at_planck_scale: bool, pub radiation: Spectrum,
    pub physics_boundary: Option<PhysicsBoundary>, // Standard mód
    pub bounce_occurred: bool, // Norbi mód
    pub baby_universe: Option<BabyUniverseState>, // Norbi mód
}
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct BabyUniverseState {
    pub scale_factor: f64, pub expansion_rate: f64,
    pub internal_density: f64, pub edge_breakup_rate: f64,
    pub total_energy: f64, pub age: f64,
}
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Spectrum {
    pub frequencies: Vec<f64>, pub intensities: Vec<f64>,
    pub temperature: f64, pub total_power: f64,
}
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct SimulationResults {
    pub schema_version: String, // "2.0" – verziókompatibilitás
    pub config: SimulationConfig,
    pub timeline: Vec<TimeStep>,
    pub evaporation_complete: bool,
}
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TimeStep {
    pub time: f64, pub mass: f64, pub temperature: f64,
    pub entropy: f64, pub spectrum: Spectrum, pub interior:
InteriorState,
}
```

21. Python–Rust határfelület és Tauri–Python összekötés

21.1 Típusleképezés

Rust típus	Python típus
f64	float
Vec<f64>	list[float]
String / &str	str
SimulationResults	str (JSON) — Python újra parseol
SimulationError	RuntimeError — PyO3 konvertálja
Option<T>	T None

21.2 Tauri–Python összekötés

A Tauri backend a Rust magot közvetlenül hívja. A Python elemző réteg (PCA, Qiskit, reverse engineering) egy subprocess-ként fut, JSON-on kommunikálva:

```
// tauri-app/src-tauri/src/python_bridge.rs
use std::process::{Command, Stdio};
use std::io::Write;

pub fn run_python_analysis(simulation_json: &str,
                           original_hash: &str) -> Result<String,
String> {
    let mut child = Command::new("python")
        .args(["-m", "python.reverse_engineer", "--stdin"])

        .stdin(Stdio::piped()).stdout(Stdio::piped()).stderr(Stdio::piped())
        .spawn().map_err(|e| format!("Python indítási hiba: {e}"))?;
    let input =
format!("{}",{"data":{"simulation_json":simulation_json,"hash":{"original_hash":original_hash}}});
    child.stdin.as_mut().unwrap().write_all(input.as_bytes())
        .map_err(|e| format!("Írási hiba: {e}"))?;
    let out = child.wait_with_output()
        .map_err(|e| format!("Python futási hiba: {e}"))?;
    if out.status.success() {
        Ok(String::from_utf8_lossy(&out.stdout).to_string())
    } else {
        Err(String::from_utf8_lossy(&out.stderr).to_string())
    }
}
```

22. Checkpoint és adatperzisztencia

Hosszú futású szimulációknál a checkpoint rendszer rendszeres időközönként menti az állapotot. Ha a program leáll, onnan folytatható ahol abbahagyta.

```
// core/src/time_evolution/checkpoint.rs
#[derive(Debug, Serialize, Deserialize)]
pub struct Checkpoint {
```

```

pub schema_version: String,    // "2.0"
pub simulation_id: String,     // UUID
pub created_at: u64,           // Unix timestamp
pub config: SimulationConfig,
pub current_time: f64,         // hol tart
pub current_mass: f64,
pub timeline_so_far: Vec<TimeStep>,
pub interior_state: InteriorState,
}

impl Checkpoint {
    pub fn save(&self, path: &Path) -> Result<(), SimulationError> {
        let bytes = rmp_serde::to_vec(self)
            .map_err(|e|
SimulationError::CheckpointFailed(e.to_string()))?;
        // Atomikus írás: tmp fájl → átnevezés – ha leáll, régi
checkpoint megmarad
        let tmp = path.with_extension("tmp");
        fs::write(&tmp, &bytes)?;
        fs::rename(tmp, path)?;
        Ok(())
    }

    pub fn load(path: &Path) -> Result<Self, SimulationError> {
        let bytes = fs::read(path)?;
        rmp_serde::from_slice(&bytes)
            .map_err(|e|
SimulationError::CheckpointFailed(e.to_string()))
    }
}

// Adatformátum migráció v1 → v2
pub fn migrate_checkpoint(old: &[u8]) -> Result<Checkpoint,
SimulationError> {
    let version: serde_json::Value = rmp_serde::from_slice(old)?;
    match version["schema_version"].as_str().unwrap_or("1.0") {
        "1.0" => migrate_v1_to_v2(old),
        "2.0" => rmp_serde::from_slice(old).map_err(Into::into),
        v => Err(SimulationError::InvalidPhysicalState {
            reason: format!("Ismeretlen checkpoint verzió: {v}") }),
    }
}

```

23. Verziókezelés — Cargo.toml és pyproject.toml

```

# Cargo.toml (workspace root)
[workspace]
members = ["core", "tauri-app/src-tauri"]
resolver = "2"
[workspace.package]
version = "2.0.0"
edition = "2021" # Minimális Rust verzió: 1.75.0
[workspace.dependencies]
serde = { version = "1.0", features = ["derive"] }

```

```

serde_json          = "1.0"
rmp-serde           = "1.1"      # MessagePack – checkpoint fájlhoz
thiserror           = "1.0"      # Hibatípus makró
pyo3                 = { version = "0.21", features = ["extension-module"]
}
approx              = "0.5"      # Lebegőpontos közelítő egyenlőség
uuid                 = { version = "1.0", features = ["v4"] }
tracing              = "0.1"      # Strukturált logging
tracing-subscriber   = "0.3"
criterion            = "0.5"      # Benchmarkok

```

```

# pyproject.toml
[build-system]
requires          = ["maturin>=1.5"]
build-backend      = "maturin"
[project]
name              = "black-hole-simulator"
version           = "2.0.0"
requires-python    = ">=3.11"
dependencies       = [
    "numpy>=1.26", "scipy>=1.12", "pandas>=2.2",
    "qiskit>=1.0", "scikit-learn>=1.4", "rich>=13.0",
]
[project.optional-dependencies]
dev = ["pytest>=8.0", "pytest-cov>=5.0",
    "ruff>=0.4", "mypy>=1.9", "hypothesis>=6.0"]
[tool.maturin]
python-source      = "python"
module-name        = "black_hole_core"

```

24. Logging és observability

```

// Rust – tracing könyvtár
use tracing::{info, warn, error, debug, instrument};
#[instrument(skip(black_hole), fields(mass = black_hole.mass()))]
pub fn evolve_one_step(black_hole: &mut dyn BlackHoleTrait,
    dt: f64) -> Result<StepResult, SimulationError> {
    debug!(dt, "Időlépés kezdete");
    let t_h = black_hole.hawking_temperature()?;
    if t_h > 1e30 { warn!(temperature=t_h, "Elpárlás közelít"); }
    let delta_m = -(black_hole.hawking_power()? / (C*C)) * dt;
    black_hole.update_mass(delta_m)?;
    info!(mass=black_hole.mass(), temperature=t_h, "Lépés kész");
    Ok(StepResult { delta_mass: delta_m })
}
// Logging init – JSON formátum, parseolható
fn init_logging() {
    tracing_subscriber::fmt().json().with_max_level(Level::INFO).init();
}

```

Log szint	Mit logolunk?
ERROR	NaN értékek, pánik, nem kezelt hibák — leállítják a szimulációt
WARN	Extrém fizikai értékek, Planck-határ közeledés, lassú konvergencia
INFO	Minden időlépés (tömeg, hőmérséklet, entrópia), checkpoint mentések
DEBUG	Numerikus köztes értékek — csak fejlesztés közben bekapcsolva

25. Teljesítménytesztek (Benchmarks)

```
// core/benches/simulation_bench.rs - Criterion
fn bench_hawking_temperature(c: &mut Criterion) {
    c.bench_function("hawking_temp_sun_mass", |b| {
        b.iter(|| hawking_temperature(M_SUN).unwrap())
    });
}

fn bench_full_evaporation_planck(c: &mut Criterion) {
    // Planck-tömeg: leggyorsabb elvégezhető teljes szimuláció
    c.bench_function("planck_mass_evaporation", |b| {
        b.iter(|| {
            let mut bh = SchwarzschildBlackHole::new(M_PLANCK).unwrap();
            let engine = HawkingEngine::new();
            while bh.mass() > M_PLANCK * 0.001 {
                let _ = engine.evolve_step(&mut bh, T_PLANCK);
            }
        })
    });
}

criterion_group!(benches, bench_hawking_temperature,
                  bench_full_evaporation_planck);
criterion_main!(benches);
```

Benchmark	Elfogadható idő
hawking_temperature()	< 100 ns
Planck-tömeg teljes elpárlás	< 50 ms
1000 bins spektrum	< 1 ms
Teljes Hold/4 szimuláció	< 30 perc

26. A scripts/ mappa tartalma

```
scripts/
├─ validate_results.py      # CI validátor
├─ benchmark_compare.py     # Két szimuláció összehasonlítása
├─ checkpoint_inspect.py    # Checkpoint megtekintése
```

```
|— export_csv.py          # Timeline CSV export
|— setup_dev.sh           # Fejlesztői környezet egy lépésben
```

```
# scripts/validate_results.py – amit a CI futtat
import json, sys
from pathlib import Path

def validate(path: str) -> list[str]:
    errors = []
    data = json.loads(Path(path).read_text())
    tl = data['timeline']
    # 1. Tömeg monoton csökken
    masses = [s['mass'] for s in tl]
    if any(masses[i] > masses[i-1] for i in range(1, len(masses))):
        errors.append('HIBA: A tömeg nem monoton csökkenő')
    # 2. Entrópia nem negatív
    if any(s['entropy'] < 0 for s in tl):
        errors.append('HIBA: Negatív entrópia')
    # 3. Hőmérséklet monoton nő
    temps = [s['temperature'] for s in tl]
    if any(temps[i] < temps[i-1] for i in range(1, len(temps))):
        errors.append('HIBA: Hőmérséklet nem monoton növekvő')
    # 4. Nincs NaN / Inf
    for s in tl:
        for k,v in s.items():
            if isinstance(v,float) and (v!=v or abs(v)==float('inf')):
                errors.append(f'HIBA: NaN/Inf: {k} t={s["time"]:.3e}s')
    # 5. Schema verzió
    if data.get('schema_version') != '2.0':
        errors.append(f'HIBA: Váratlan schema:
{data.get("schema_version")}')
    return errors

if __name__ == '__main__':
    errs = []
    for path in sys.argv[1:]:
        e = validate(path)
        errs.extend(e)
        print(f'{path}: {"OK" if not e else str(len(e))+" HIBA"}')
        for x in e: print(f'  {x}')
    sys.exit(1 if errs else 0)
```

27. Végleges mappaszerkezet — minden javítással

```
black-hole-simulator/
|— Cargo.toml          # Workspace + pontos verziók (v23)
|— pyproject.toml      # Python projekt (v23)
|— .github/workflows/
|   |— rust-tests.yml
|   |— python-tests.yml
```

```

└─ integration.yml
└─ core/
  └─ Cargo.toml
  └─ benches/simulation_bench.rs # Criterion benchmarkok (v25)
  └─ src/
    └─ lib.rs # PyO3 + panic handler (v19)
    └─ error.rs # SimulationError (v19)
    └─ types.rs # Teljes adatséma (v20)
    └─ constants.rs
    └─ black_hole/{mod,schwarzschild,kerr,thermodynamics}.rs
    └─ interior/{mod,standard,norbi,baby_universe,cauchy}.rs
    └─ radiation/{mod,hawking_engine,spectrum,soft_hair}.rs
    └─ quantum/{mod,lqc,island,complexity}.rs
    └─ time_evolution/
      └─ integrator.rs # RK45 + tracing (v24)
      └─ checkpoint.rs # Checkpoint (v22)
    └─ tests/
      └─ test_schwarzschild.rs
      └─ test_hawking.rs
      └─ test_lqc.rs
      └─ test_interior.rs
      └─ test_spectrum.rs
      └─ test_edge_cases.rs # Határesetek (v19)
  └─ python/
    └─ logging_config.py # JSON logging (v24)
    └─ config.py
    └─ information_packet.py
    └─ reverse_engineer.py
    └─ comparator.py
    └─ gec_model.py
    └─ quantum_sim.py
    └─ tests/{test_information,test_reverse_eng,test_comparator}.py
  └─ tauri-app/
    └─ src-tauri/src/
      └─ main.rs
      └─ commands.rs
      └─ python_bridge.rs # Subprocess híd (v21)
    └─ src/components/{Dashboard,ConfigPanel,InteriorView,...}.tsx
  └─ scripts/
    └─ validate_results.py # CI validátor (v26)
    └─ benchmark_compare.py
    └─ checkpoint_inspect.py
    └─ export_csv.py
    └─ setup_dev.sh
  └─ docs/{physics,api}/

```

28. Javítások összefoglalása

Azonosított hiányosság	Megoldás
Nincs hibakezelés	SimulationError enum, Result<T,E> mindenütt, pánik handler — 19. fej.

Adatséma részleges	Teljes Rust struktúrák: Particle, InteriorState, Spectrum, stb. — 20. fej.
PyO3 határfelület nem részletezett	Típusleképezési táblázat, pánik kezelés — 21. fej.
Tauri-Python összekötés hiányzik	python_bridge.rs subprocess JSON kommunikáció — 21. fej.
Nincs adatperzisztencia	MessagePack checkpoint, atomikus írás, migráció — 22. fej.
Verziókezelés hiányzik	Teljes Cargo.toml + pyproject.toml pontos verziókkal — 23. fej.
Nincs logging	Rust tracing + Python JSON logging, szintrendszer — 24. fej.
Nincs benchmark	Criterion benchmarkok, elfogadható határok — 25. fej.
scripts/ mappa üres volt	validate_results.py és többi szkript leírva — 26. fej.
Fizikai határesetek nem tesztelve	test_edge_cases.rs — nulla tömeg, NaN, negatív — 19. fej.

Az architektúra most hibakezelés, adatperzisztencia, logging, benchmarkok és teljes adatséma szempontjából is kifogástalan. Következő lépés: a tudományos háttér formalizálása.

III. RÉSZ — Tudományos háttér

1. A kutatási kérdés

Ez a szimulációs projekt egy numerikus gondolkísérlet — nem helyettesíti a valódi fizikai mérést, de lehetővé teszi hogy számszerűen megvizsgáljuk: konzisztens-e a Norbi-hipotézis az ismert fizikával, és ha igen, milyen mérhető különbséget produkál a standard modellhez képest.

Kutatási kérdés: Ha egy fekete lyuk belsejében a kvantumvisszapattanás során egy táguló bébiuniverzum keletkezik, és a tágulás szélén szétszakadó anyag a Hawking-sugárzás forrása — akkor ez a sugárzás szignifikánsan különbözik-e a standard vákuumfluktuációs modelltől, és a kijövő sugárzásból visszakövetkeztethető-e a beejtett információ struktúrája?

2. Formális hipotézis

2.1 A Norbi-hipotézis — H_1

A hipotézist három egymásra épülő állításra bontjuk, amelyek mindegyike önállóan is tesztelhető:

$H_{1.A}$ — A sugárzás forrása

$H_{1.A}$: A Hawking-sugárzás spektruma szignifikánsan eltér a tisztán termikus Planck-spektrumtól, ha a sugárzás forrása a belső bébiuniverzum szélén
szétszakadó anyag – nem pedig a vákuumfluktuáció az eseményhorizonton.

Mérhető következmény:

```
|S_norbi(v) - S_planck(v)| /  $\sigma_{noise}$  > 3.0  
ahol  $\sigma_{noise}$  a kvantumfluktuációs zaj amplitúdója  
(3-sigma küszöb = szignifikáns különbség)
```

$H_{1.B}$ — Információmegmaradás

$H_{1.B}$: A Norbi-módban a beejtett információcsomag nagyobb arányban nyerhető vissza a kijövő sugárzásból, mint a standard módban.

Mérhető következmény:

```
similarity_norbi - similarity_standard >  $\delta_{threshold}$   
ahol  $\delta_{threshold}$  = 0.05 (5% javulás már szignifikáns)  
similarity = a rekonstruált és az eredeti hash bit-egyezési aránya
```

$H_{1.C}$ — Bébiuniverzum lenyomata

$H_{1.C}$: A kijövő Hawking-sugárzás spektrumának időbeli változása szignifikáns
korrelációt mutat egy táguló bébiuniverzum CMB spektrumával.

Mérhető következmény:

```
|cross_correlation(S_hawking(t), S_CMB(t))| > 0.3  
(Pearson-korreláció, ahol 0 = nincs, 1 = tökéletes)
```

3. Null hipotézis — H_0

A null hipotézis azt állítja, hogy a Norbi-modell nem ad mérhető különbséget a standard modellhez képest. Ezt kell megcáfolni ahhoz, hogy a hipotézis megerősítést kapjon.

H₀.A: A két modell sugárzási spektruma nem különbözik szignifikánsan.
 $|S_{\text{norbi}}(v) - S_{\text{planck}}(v)| / \sigma_{\text{noise}} \leq 3.0$ (minden v -re)

H₀.B: Az információ visszanyerési arány nem javul Norbi módban.
 $\text{similarity}_{\text{norbi}} - \text{similarity}_{\text{standard}} \leq 0.05$

H₀.C: Nincs szignifikáns korreláció a sugárzás és a CMB spektrum között.
 $|\text{cross_correlation}| \leq 0.3$

Ha mindhárom null hipotézist el tudjuk vetni:
 → A Norbi-hipotézis numerikusan konzisztens és megkülönböztethető

Ha egyik sem vethető el:
 → A két modell numerikusan ekvivalens – a hipotézis jelenlegi formájában
 nem tesztelhető szimulációval

Ha részben vethető el:
 → Részleges megerősítés – pontosan jelöljük melyik állítás áll meg

4. Validációs stratégia

4.1 1. lépés — Standard modell validálása ismert eredményekkel

Mielőtt a Norbi-hipotézist teszteljük, a standard modellnek pontosan reprodukálnia kell az összes ismert analitikus eredményt. Ha ez nem teljesül, a szimulációban hiba van.

Validálandó eredmény			Elvárt érték
Validálandó eredmény	Elvárt érték	Forrás [hivatkozás]	
T _H (M _{Nap})	$\approx 6.17 \times 10^{-8} \text{ K}$	[HAW74]	
r _s (M _{Nap})	$\approx 2954 \text{ m}$	[SCH16]	
t _{evap} $\propto M^3$ arány	2×10^{67} év napnyi tömegnél	[HAW75]	
Spektrum csúcs – Wien-törvény	$v_{\text{max}} = 5.879 \times 10^{10} \cdot T$	[PLA00]	
Entrópia skálázás $S \propto M^2$	Kétszeres tömeg → 4× entrópia	[BEK73]	
Page-görbe alakja	Csúcs t _{Page} $\approx t_{\text{evap}}/2$ -nél	[PAG93]	
LQC visszapattanás feltétele	$H^2 = 0$ ha $\rho = \rho_{\text{Planck}}$	[ASH06]	

4.2 2. lépés — A két modell összehasonlítása

Ha a standard modell validált, lefuttatjuk mindkét módban a szimulációt azonos bemeneti paraméterekkel, és összehasonlítjuk a kimeneteket a 3. fejezetben definiált metrikákkal.

4.3 3. lépés — Szenzitivitás-analízis

A szimulációs paraméterek bizonytalansága befolyásolja az eredményt. Megvizsgáljuk hogy a következtetések robusztusak-e ha a paramétereket $\pm 10\%$ -kal változtatjuk:

Vizsgált paraméterek:

- Kezdeti tömeg (M_0): $\times 0.5$, $\times 1$, $\times 2$, $\times 5$
- Időlépés mérete (dt): adaptív, $\times 0.1$, $\times 10$
- Spektrum felbontás: 100, 1000, 10000 bin
- Bébiuniverzum kezdeti skálafaktor (a_0): 1_P , $10 \cdot 1_P$, $100 \cdot 1_P$

Elvárt viselkedés:

- A standard modell eredményei nem érzékenyek ezekre
- A Norbi-modell eredményei gyengén érzékenyek a_0 -ra
- Ha erősen érzékeny: a modell nem robusztus $\rightarrow$ jelölni kell

5. A szimuláció korlátai

Egy komoly kutatásban a korlátokat explicit módon kell közölni. Ezek nem hibák — hanem az egyszerűsítések becsületes leírása.

5.1 Elméleti korlátok

Korlát	Részletek és következmény
AdS/CFT érvényessége	Az island formula és a HKLL rekonstrukció anti-de Sitter téridőre van bizonyítva. A mi univerzumunk de Sitter típusú (pozitív kozmológiai állandóval). Az eredmények átvihetők-e? Részlegesen — de ez nem teljes bizonyíték.
Schwarzschild közelítés	A v2.0 nem forgó fekete lyukat modellez. A valóságban minden fekete lyuk forog (Kerr). A Kerr-géometria belső struktúrája lényegesen különbözik — ez a v3.0 feladata.
LQC egyszerűsítés	A Loop Quantum Cosmology egyenlete homogén, izotróp téridőre van levezetve. A fekete lyuk belseje erősen inhomogén. Az egyenlet alkalmazása itt közelítés.
Kvantumgravitáció hiánya	A szingularitás közelében sem az általános relativitáselmélet, sem a kvantummechanika önmagában nem érvényes. Teljes kvantumgravitációs elmélet nélkül a visszapattanás modellezése hipotézis.
Véges numerikus pontosság	A lebegőpontos számítások 15-16 jegy pontossággal dolgoznak. Hosszú szimulációknál a hibák felhalmozódhatnak. A szenzitivitás-analízis ezt vizsgálja.

5.2 Mit állíthatunk az eredmények alapján?

ÁLLÍTHATÓ:

- ✓ A Norbi-hipotézis numerikusan konzisztens (vagy nem konzisztens) az LQC és a Schwarzschild-geometria keretein belül
- ✓ A két modell spektruma különbözik (vagy nem) az adott paramétereken
- ✓ Az information visszanyerési arány jobb (vagy nem) Norbi módban

NEM ÁLLÍTHATÓ:

- X Hogy a Norbi-hipotézis fizikailag igaz
- X Hogy a valódi fekete lyukak így viselkednek
- X Hogy az eredmények forgó fekete lyukakra is érvényesek (v3.0 előtt)
- X Hogy az AdS/CFT alapú eredmények de Sitter téridőben is érvényesek

6. Irodalomjegyzék

Az alábbi hivatkozások a dokumentumban szereplő összes fizikai eredmény forrásait tartalmazzák. A hivatkozási kód [XXXév] formátumú.

6.1 Alapvető elméleti fizika

[SCH16] Schwarzschild, K. (1916). Über das Gravitationsfeld eines Massenpunktes nach der Einsteinschen Theorie. Sitzungsberichte der Königlich Preussischen Akademie der Wissenschaften, 7, 189–196.

[EIN15] Einstein, A. (1915). Die Feldgleichungen der Gravitation. Sitzungsberichte der Preussischen Akademie der Wissenschaften zu Berlin, 844–847.

[KER63] Kerr, R. P. (1963). Gravitational Field of a Spinning Mass as an Example of Algebraically Special Metrics. Physical Review Letters, 11(5), 237–238.

[PLA00] Planck, M. (1900). Zur Theorie des Gesetzes der Energieverteilung im Normalspectrum. Verhandlungen der Deutschen Physikalischen Gesellschaft, 2, 237–245.

6.2 Fekete lyuk termodinamika és Hawking-sugárzás

[BEK73] Bekenstein, J. D. (1973). Black holes and entropy. Physical Review D, 7(8), 2333–2346.

[HAW74] Hawking, S. W. (1974). Black hole explosions? Nature, 248, 30–31.

[HAW75] Hawking, S. W. (1975). Particle creation by black holes. Communications in Mathematical Physics, 43(3), 199–220.

[BAR73] Bardeen, J. M., Carter, B., & Hawking, S. W. (1973). The four laws of black hole mechanics. Communications in Mathematical Physics, 31(2), 161–170.

6.3 Információs paradoxon és holografikus elv

[PAG93] Page, D. N. (1993). Information in black hole radiation. *Physical Review Letters*, 71(23), 3743–3746.

[MAL97] Maldacena, J. (1997). The Large N limit of superconformal field theories and supergravity. *International Journal of Theoretical Physics*, 38, 1113–1133.

[HAW16] Hawking, S. W., Perry, M. J., & Strominger, A. (2016). Soft Hair on Black Holes. *Physical Review Letters*, 116(23), 231301.

[ALM19] Almheiri, A., Engelhardt, N., Marolf, D., & Maxfield, H. (2019). Entanglement Wedge Reconstruction and the Information Paradox. *Journal of High Energy Physics*, 2019(12), 63.

[ALM20] Almheiri, A., Mahajan, R., Maldacena, J., & Zhao, Y. (2020). The Page curve of Hawking radiation from semiclassical geometry. *Journal of High Energy Physics*, 2020(3), 149.

6.4 Kvantumgravitáció és Loop Quantum Cosmology

[ASH06] Ashtekar, A., & Pawłowski, T., & Singh, P. (2006). Quantum Nature of the Big Bang: Improved dynamics. *Physical Review D*, 74(8), 084003.

[BOJ01] Bojowald, M. (2001). Absence of a Singularity in Loop Quantum Cosmology. *Physical Review Letters*, 86(23), 5227–5230.

[SMO92] Smolin, L. (1992). Did the universe evolve? *Classical and Quantum Gravity*, 9(1), 173–191.

[HAR83] Hartle, J. B., & Hawking, S. W. (1983). Wave function of the universe. *Physical Review D*, 28(12), 2960–2975.

6.5 Holografikus komplexitás és kvantumhibajavítás

[SUS14] Susskind, L. (2014). Computational Complexity and Black Hole Horizons. *arXiv:1402.5674*.

[BRO16] Brown, A. R., Roberts, D. A., Susskind, L., Swingle, B., & Zhao, Y. (2016). Complexity, Action, and Black Holes. *Physical Review D*, 93(8), 086006.

[ALM15] Almheiri, A., Dong, X., & Harlow, D. (2015). Bulk Locality and Quantum Error Correction in AdS/CFT. *Journal of High Energy Physics*, 2015(4), 163.

[HKL06] Hamilton, A., Kabat, D., Lifschytz, G., & Lowe, D. A. (2006). Local bulk operators in AdS/CFT: A boundary view of horizons and locality. *Physical Review D*, 73(8), 086003.

6.6 Analóg fekete lyukak és kísérleti vonatkozások

[STI19] Steinhauer, J. (2019). Observation of thermal Hawking radiation and its temperature in an analogue black hole. *Nature Physics*, 2019.

[GOU22] Google Quantum AI (2022). Traversable wormhole dynamics on a quantum processor. *Nature*, 612, 51–55.

7. A kutatási terv összefoglalása

Elem	Tartalom
------	----------

Kutatási kérdés	Konzisztens-e a Norbi-hipotézis az ismert fizikával, és produkál-e mérhető különbséget?
H_1 – Norbi-hipotézis	Három falszifikálható állítás: spektrum különbség, információ visszanyerés, CMB korreláció
H_0 – Null hipotézis	A két modell numerikusan ekvivalens — ezt kell megcáfolni
Validáció 1. lépés	Standard modell reprodukálja az összes ismert analitikus eredményt
Validáció 2. lépés	Mindkét mód lefuttatása, 3 metrika összehasonlítása
Validáció 3. lépés	Szenzitivitás-analízis: robusztusak-e az eredmények?
Elméleti korlátok	AdS/CFT érvényessége, Schwarzschild közelítés, LQC inhomogenitás — mind explicit jelölve
Mit állíthatunk	Numerikus konzisztencia — nem fizikai igazság
Irodalomjegyzék	26 hivatkozás, minden állítás alátámasztva

FONTOS: Ez a kutatás numerikus szimulációra épül, nem valódi fizikai mérésre. Az eredmények érvényessége az alkalmazott elméleti modellek érvényességi határain belül értelmezendő. A korlátokat minden esetben explicit módon jelöljük.

A kutatási terv most tudományosan becsületes: pontosan megmondja mit vizsgál, hogyan, milyen feltételek mellett, és mit lehet — és mit nem lehet — az eredmények alapján állítani.
